

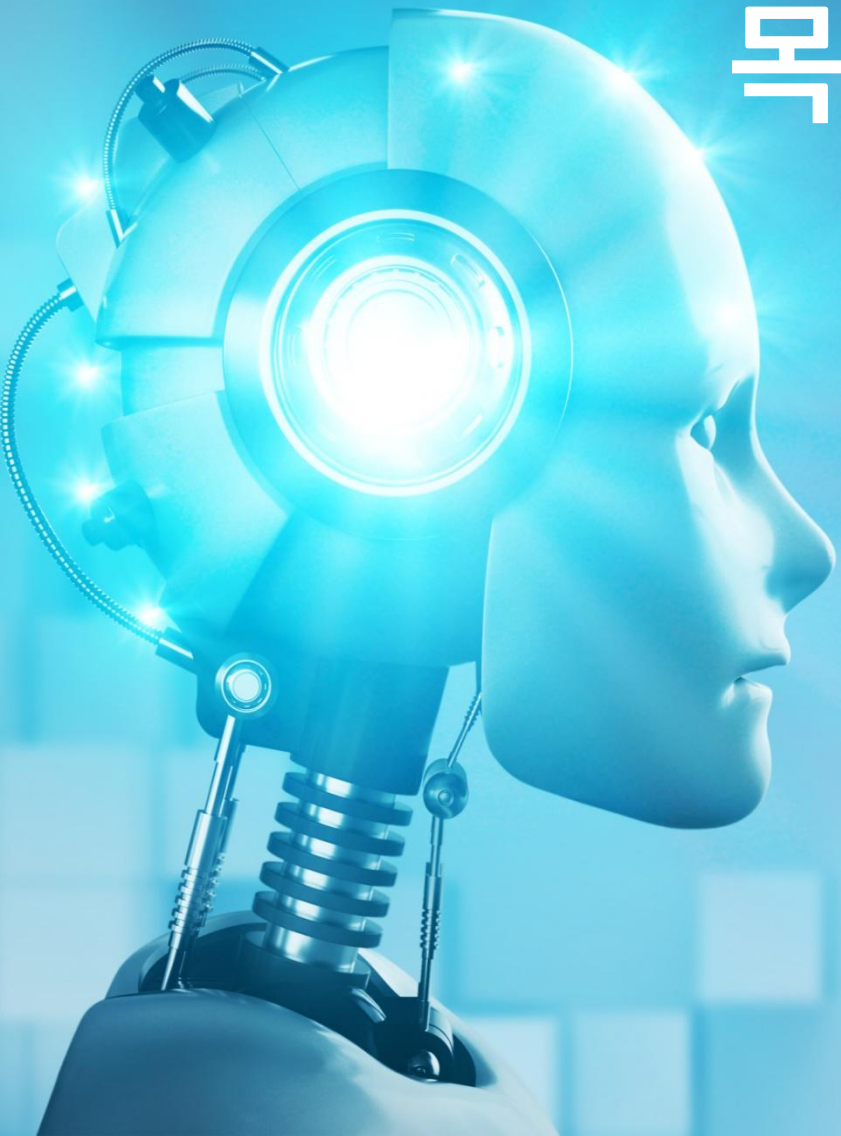
인공지능(AI) 개발자 양성 과정

4
차시

딥러닝 기본

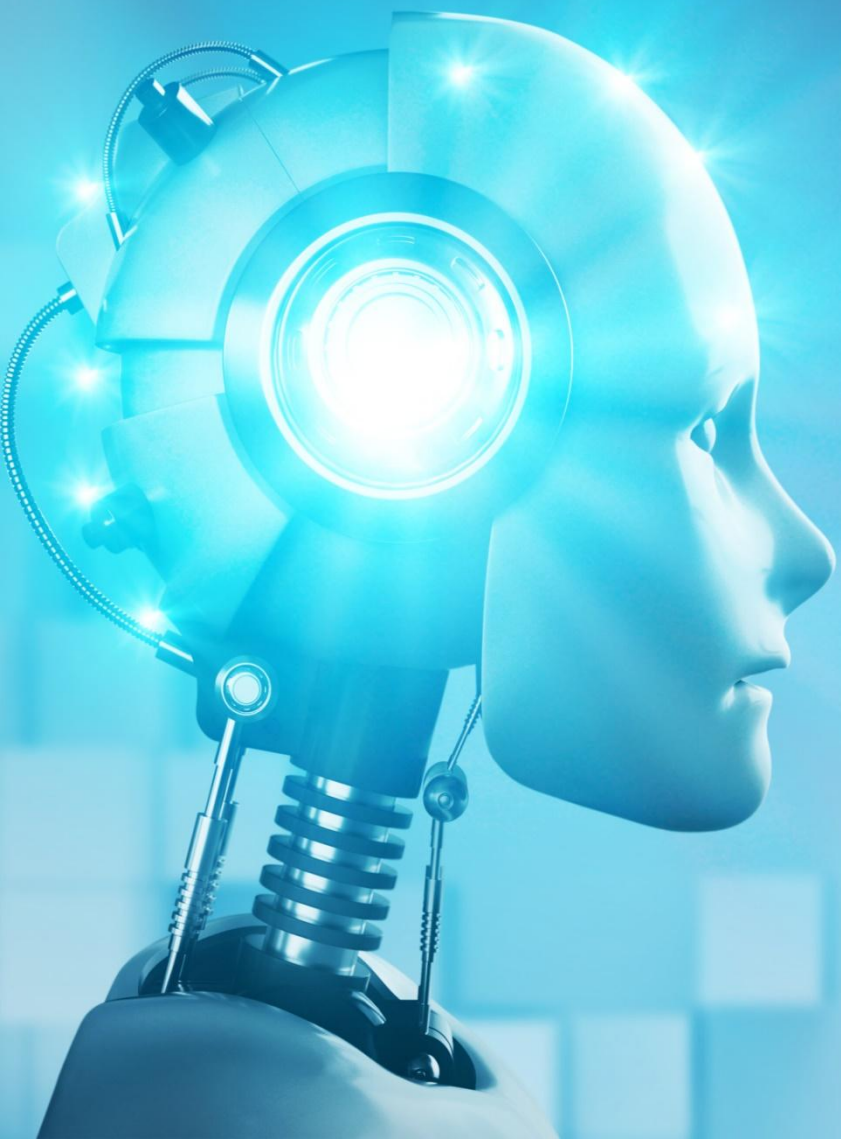
휴먼교육센터 안홍근 강사





목차 Contents

1. 딥러닝 기초



1

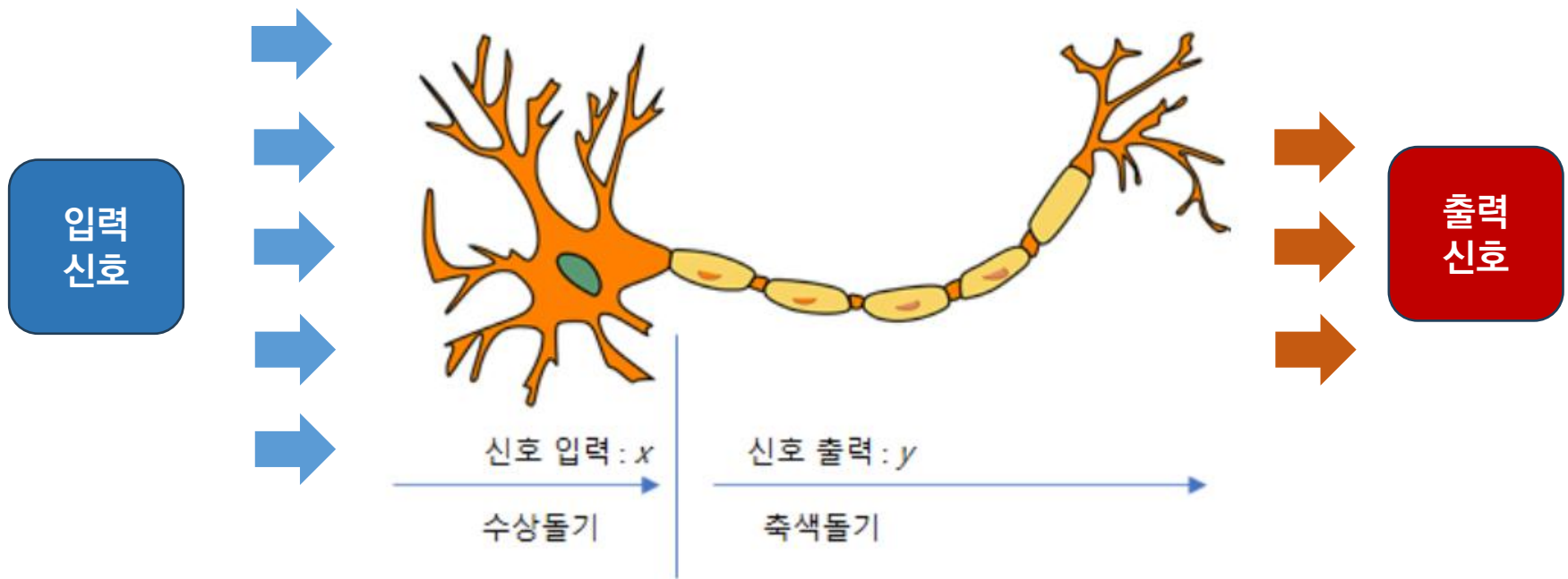
딥러닝 기초

1. 신경망
2. 활성화 함수
3. 텐서플로우와 케라스
4. 합성곱 신경망(CNN)
5. RNN과 LSTM
6. 생성적 적대 신경망(GAN)

1. 신경망

1.1 인간의 신경계와 신경세포

- ☑ 인간의 신경계는 복잡하게 연결된 뉴런과 시냅스로 구성 (cf. 시냅스 : 뉴런 사이의 틈), 수많은 뉴런들이 연결되어 정보전달
- ☑ 수상돌기와 축색돌기는 각각 입력과 출력의 역할을 수행하며, 이는 머신러닝의 인공 신경망에도 동일하게 적용됨



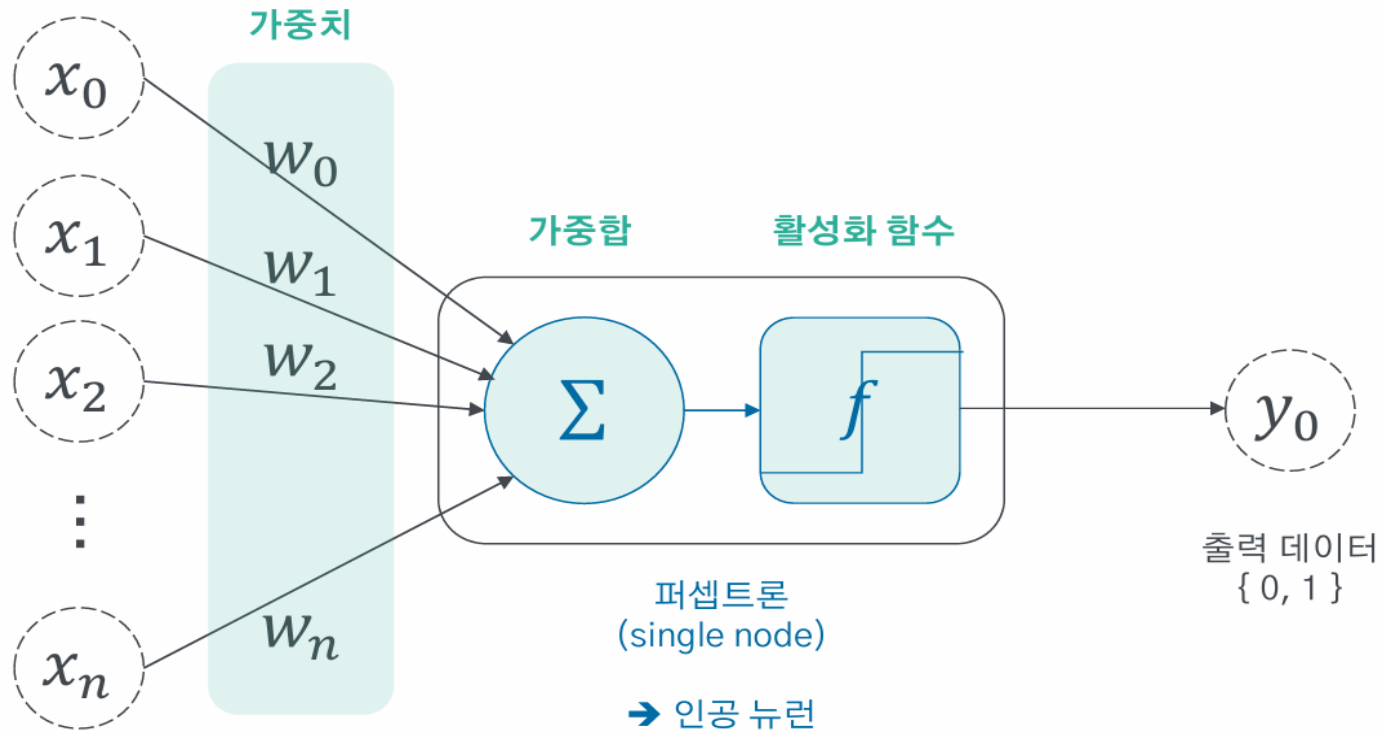
* 수상돌기 : 다른 뉴런의 신호를 받아들이는 부분으로 **입력층**의 역할을 수행

* 축색돌기 : 세포체에서 처리된 정보를 다른 뉴런으로 전달하는 부분으로 **출력층** 역할을 수행

1. 신경망

1.2 인공 신경망(퍼셉트론)

- ☑ **인공신경망**은 인간의 뉴런을 모방하여 **인간 두뇌와 동일한 방식**으로 **복잡한 문제를 해결**하는 것에 초점을 맞춤
- ☑ 인공신경망은 수많은 함수를 서로 연결하여 복잡한 정보를 처리
- ☑ 입력값을 가중치와 곱하고 그 결과를 전부 더한 후, 활성화 함수에 입력하여 최종 출력으로 변환

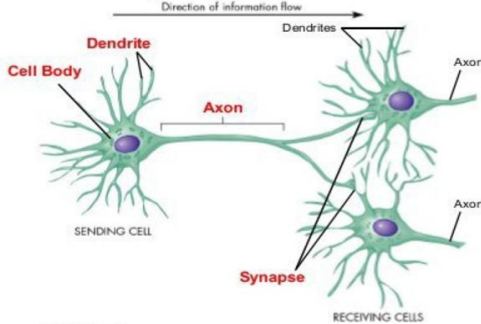
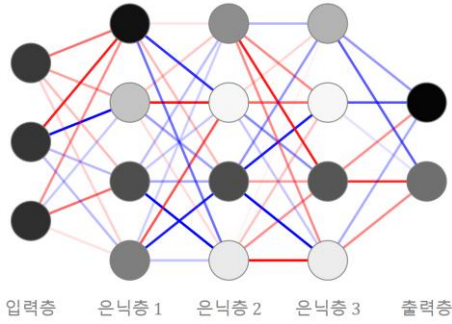


* (활성화 함수의 예) 시그모이드(sigmoid) 함수 : 입력이 0.5 보다 작으면 0으로 분류, 0.5보다 크면 1로 분류

1. 신경망

1.3 인간 신경계 vs 인공 신경망

- ☑ 인공신경망은 인간의 뇌를 모방하여 만들어졌기 때문에 수상돌기와 축색돌기의 역할을 하는 부분이 존재

항목	인간 신경계	인공 신경망
입력층	수상돌기 (다양한 형태의 신호 수신)	입력노드 (숫자 형태의 데이터 수신)
출력층	축색돌기 (다른 뉴런이나 기관으로 신호 전달)	출력 노드(예측 값 또는 분류 결과 출력)
정보전달 방식	전기 신호	수치 계산
학습방식	경험을 통한 시냅스 가중치 변화	데이터 기반 학습 알고리즘 (역전파 등)
이미지	 The diagram illustrates a biological neuron. It shows a 'SENDING CELL' on the left with 'Dendrites' and a 'Cell Body'. An 'Axon' extends from the cell body towards 'RECEIVING CELLS' on the right. A 'Synapse' is shown where the axon of the sending cell meets the dendrites of the receiving cells. An arrow at the top indicates the 'Direction of information flow' from left to right.	 The diagram shows an artificial neural network with four layers of nodes. The first layer is the '입력층' (Input Layer) with 4 black nodes. The next three layers are '은닉층 1' (Hidden Layer 1) with 3 grey nodes, '은닉층 2' (Hidden Layer 2) with 3 white nodes, and '은닉층 3' (Hidden Layer 3) with 3 grey nodes. The final layer is the '출력층' (Output Layer) with 2 black nodes. Nodes in adjacent layers are connected by lines of various colors (red, blue, green, yellow).

2. 활성화 함수

2.1 활성화 함수

- ☑ 인공 신경망에서 출력값을 결정하는 데 사용되는 함수
- ☑ 입력 신호를 변환하여 비선형성을 부여하고 신경망이 복잡한 데이터 패턴을 학습할 수 있도록 지원

활성화 함수 특징

- 비선형성 : 신경망이 비선형 문제를 해결할 수 있게 함
- 출력값 범위 제한 : 학습 안정화에 도움
- 계산 효율성 : 빠르고 효율적인 계산을 통해 모델 성능 최적화
- 미분 가능성 : 역전파를 통해 가중치 업데이트를 가능하게 함

활성화 함수 예시

- Sigmoid : 출력값을 (0, 1)로 압축
- Tanh : 출력값을 (-1, 1)로 압축
- ReLU : 음수를 0으로 설정, 양수는 그대로 출력
- Leak ReLU : ReLU에서 음수 영역에 작은 기울기를 부여
- Softmax : 다중 클래스 분류에서 확률로 변환

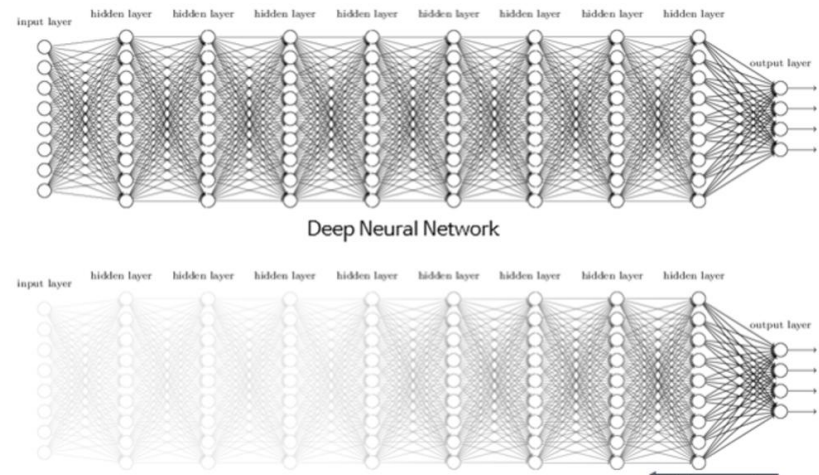
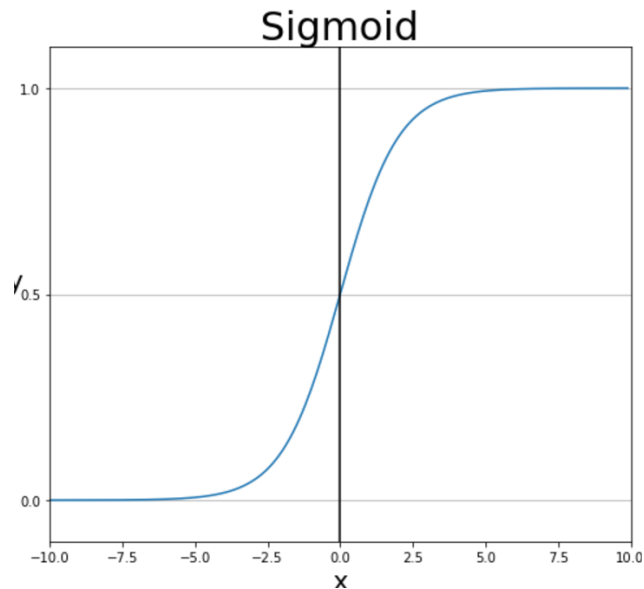
2. 활성화 함수

2.2 Sigmoid (시그모이드) 함수

- ☑ Sigmoid 함수는 Logistic 함수라고 불리기도 하며, **x의 값에 따라 S자형** 함수임
- ☑ 입력 값을 **0과 1 사이의 확률값**으로 변환하는 비선형 함수

sigmoid 함수 특징

- 신경망에서 확률적 해석이 필요한 경우 사용
- 입력 값이 매우 크거나 작을 때 기울기가 거의 0에 수렴해서 **기울기 소실문제 발생**
- 중간 영역에서만 의미 있는 기울기 제공
- 주요 사용 영역 : 이진 분류 문제의 출력층, 로지스틱 회귀



기울기 소실 문제 발생

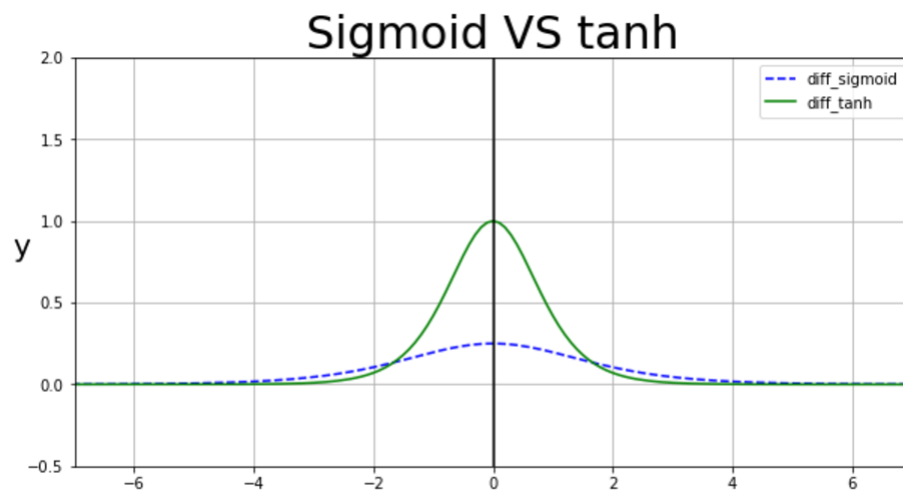
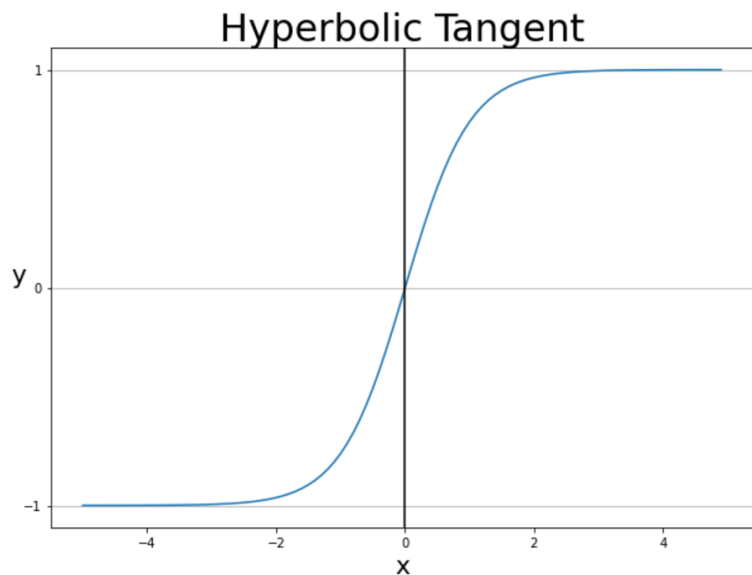
2. 활성화 함수

2.3 Tanh (하이퍼볼릭 탄젠트) 함수

☑ Tanh 함수는 입력 값을 **-1에서 1 사이로 정규화**하는 대칭적 비선형 함수

Tanh 함수 특징

- 제로 중심화된 출력으로 대칭적 특성
- 입력 신호의 상대적 중요성을 음수와 양수로 표현
- sigmoid보다 경사면이 더 가파른 중간 영역
- 대칭성으로 인해 학습 속도 개선 가능
- 주요 사용 영역 : 순환신경망(RNN), 자연어 처리 모델의 은닉층



Tanh 함수는 Sigmoid함수의 기울기 소실 문제 일부 개선

2. 활성화 함수

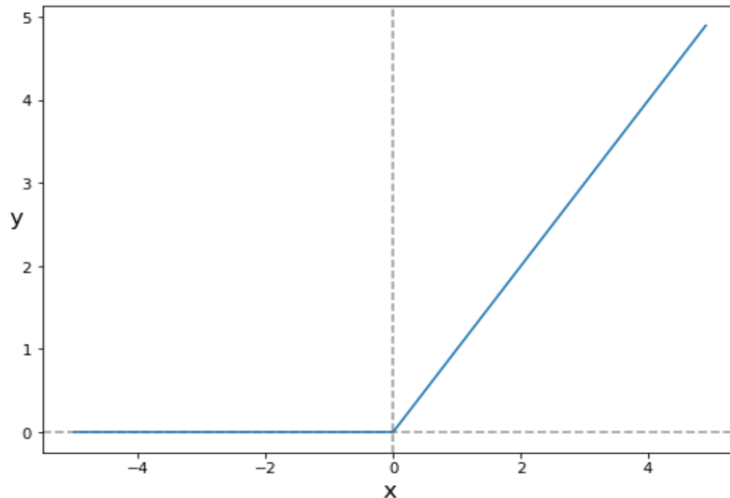
2.4 ReLU 함수

☑ ReLU 함수는 음수 입력에 대해 0을 출력, 양수 입력에 대해 선형적으로 출력(그대로 출력)

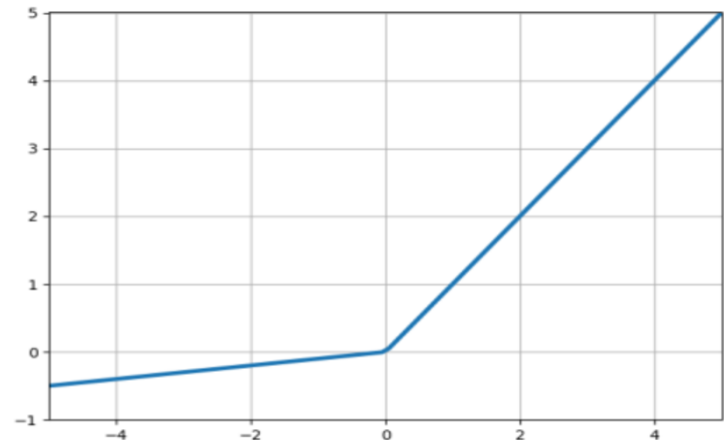
ReLU 함수 특징

- 계산의 단순성과 빠른 수렴으로 계산 복잡도가 낮아 학습 속도 향상
- 비선형성과 데이터 희소성 동시 제공
- 기울기 소실문제 해결하고 있으나, 출력값이 0에서 무한대로 커질 수 있음
- Dying ReLU : 가중치 업데이트 과정에서 가중치 합이 음수가 되면 0만 반환되어 변하지 않는 현상 발생
- 주요 사용 영역 : 이미지 분류 모델, 생성적 적대 신경망(GAN)

ReLU



Leaky ReLU



Dying ReLU 현상 보완

2. 활성화 함수

2.5 Softmax 함수

- ☑ Softmax 함수는 Sigmoid 함수와 비슷하게 **0~1사이로 변환하여 출력**하지만, 각 출력 값은 **전체 출력 값의 합이 1이 되도록 정규화됨**(마치 확률 분포처럼 해석)

Softmax 함수 특징

- 확률 분포 반환 : 출력 값의 합이 항상 1이므로, 분류 문제에서 클래스의 확률을 나타내는 데 사용
- 상대적 크기 강조 : 출력 값이 크면 더 큰 확률로 변환되며, 값이 작으면 확률이 낮아짐
- 다중 클래스 분류에 적합 : n개의 출력 값을 가지는 다중 클래스 분류 문제에서 주로 사용
- 예시 : 이미지를 보고 고양이, 강아지, 토끼를 분류할 때, 모델이 각 동물에 대한 확률 값을 출력하여 가장 확률이 높은 동물을 선택

각 클래스에 대한 점수 벡터가 다음과 같다고 가정

분류 = [A, B, C]

점수 = [2.0, 1.5, 1.0]

이 벡터에 max 함수를 적용하면 다음과 같이 나오지만 세 클래스에 대한 정보를 제공하지 않음

$\max(scores) = 2.0$

softmax 함수를 적용하면 세 클래스의 합이 1이 되어 세 클래스에 대한 유효한 확률 분포를 형성

$\text{softmax}(scores) = [0.496, 0.333, 0.170]$

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

- n = 출력층의 뉴런 수 (총 클래스 수)
- x = x 번째 클래스

2. 활성화 함수

2.6 활성화 함수 비교

구분	Sigmoid	Tanh	ReLU	Softmax
정의	입력 값을 0과 1사이로 변환	입력 값을 -1과 1 사이로 변환	음수는 0, 양수는 그대로 출력	출력 값을 확률 분포로 변환
출력범위	0 ~ 1	-1 ~ 1	0 ~ ∞ (무한대)	0 ~ 1
특징	S자형 곡선, 확률 계산에 적합	S자형 곡선, 중심이 0	음수는 0, 양수는 그대로, 계산 간단	모든 출력 값을 정규화해 확률로 변환
장점	확률 해석 가능, 간단한 계산	중심이 0이라 데이터 처리에 유리	Gradient Vanishing 문제 없음, 효율적	다중 클래스 확률 제공, 직관적 해석
단점	Gradient Vanishing 문제, 학습 속도 저하	Gradient Vanishing 문제	Dying ReLU 문제 발생	다른 출력 값에 의존
사용상황	이진 분류, 확률 계산	중심이 0인 데이터	이미지 분류에 사용	다중 클래스 분류 문제

* Gradient Vanishing (기울기 소실) 문제 : 신경망의 활성화함수의 도함수 값이 계속 곱해지다 보면 가중치에 따른 결과값의 기울기가 0이되어 버리는 문제 발생

3. 텐서플로우와 케라스

텐서플로우와 사이킷런 비교

- ☒ 텐서플로우는 딥러닝과 대규모 데이터를 다룰 때, 사이킷런은 간단하고 빠른 머신러닝 작업에 적합

항목	사이킷런(scikit-learn)	텐서플로(TensorFlow)
주요 용도	머신러닝, 간단한 모델 구현 및 분석	딥러닝, 복잡한 신경망 설계 및 학습
지원 알고리즘	다양한 머신러닝 알고리즘(선형 회귀, SVM, 랜덤 포레스트 등)	신경망 기반 모델(CNN, RNN, Transformer 등)
데이터처리 능력	소규모 데이터와 정형 데이터(표 형태의 데이터)에 적합	대규모 데이터와 비정형 데이터(이미지, 음성, 텍스트 등) 처리 가능
비선형문제 처리	비교적 간단한 비선형 문제 해결 가능	복잡한 비선형 문제를 효과적으로 처리 가능
유연성	간단하고 빠르게 사용 가능하지만 커스터마이징은 제한적	커스터마이징 가능 : 사용자 정의 레이어, 손실함수, 모델 설계 가능
주요 활용사례	데이터 분석, 예측 모델, 분류, 회귀 등 비교적 단순한 머신러닝 작업	이미지 분류, 음성 인식, 자연어 처리 등 딥러닝이 필요한 복잡한 작업

3. 텐서플로우와 케라스

3.1 텐서플로우(TensorFlow) ▶ 3.1.1 개요 (1/2)

- ☑ 텐서플로우는 **구글이 개발한 오픈소스 소프트웨어 라이브러리**이며 머신러닝과 딥러닝을 쉽게 사용할 수 있게 다양한 기능 제공
- ☑ 데이터를 통해 복잡한 수치 계산을 수행할 수 있는 강력한 도구로, **딥러닝 모델을 구축하고 훈련시키는데 사용**
- ☑ 이름에서 알 수 있듯이 텐서(다차원 배열)의 흐름을 다루는 도구

특징

- 다양한 하드웨어 플랫폼(CPU, GPU, TPU)에서 작동 가능
- 대규모 시스템부터 모바일 기기까지 다양한 플랫폼에서 사용 가능
- 계산을 방향성 그래프로 표현
- 복잡한 수학적 연산을 직관적으로 모델링

장점

- 강력한 지원 : 구글의 지원과 큰 커뮤니티 덕분에 많은 자료와 도구를 이용 가능
- 유연한 모델 구축 : 복잡한 모델부터 간단한 모델까지 다양하게 만들 수 있음
- 다양한 분야 활용 : 연구, 산업, 교육 등 여러 분야에 사용됨

단점

- 초기 학습 곡선이 가파름
- 다른 프레임워크에 비해 복잡할 수 있음
- 대규모 프로젝트에서는 설정이 복잡할 수 있음

3. 텐서플로우와 케라스

3.1 텐서플로우(TensorFlow) ▶ 3.1.1 개요 (2/2)

- ☑ 텐서(Tensor) : 쉽게 말해 **다차원 배열**. (cf. 배열 : 리스트, 행렬과 비슷한 개념), **Tensorflow**는 이러한 텐서를 다루는 라이브러리
- ☑ 딥러닝에서는 데이터를 텐서 형태로 표현하고, 텐서 연산을 통해 모델을 학습시킴

1차원 텐서

- 숫자들의 리스트 또는 배열을 나타냄
- 예를 들어, [1, 2, 3], [4.5, 5.6, 6.7]

2차원 텐서

- 숫자들의 2차원 배열을 나타냄
- 비유 : 레고 블록을 평면으로 쌓은 것
- 예를 들어, [[1, 2], [3, 4]]

3차원 텐서

- 숫자들의 3차원 배열을 나타냄
- 비유 : 레고 블록을 입체적으로 쌓은 것
- 예를 들어, 이미지 데이터에서 (높이, 너비, 채널) 형태

4차원 텐서

- 숫자들의 4차원 배열을 나타냄, 레고 블록으로 만든 여러 개의 입체 구조물을 모아 놓은 것에 비유
- 예를 들어, 이미지 데이터에서 (이미지 개수, 높이, 너비, 채널) 형태
- MNIST 데이터셋 : 60,000개의 훈련 이미지, 각 이미지가 28 x 28픽셀 크기, 흑백이미지 이므로, 전체 훈련 데이터는 (60000, 28, 28, 1) 형태의 4차원 텐서로 표현됨

3. 텐서플로우와 케라스

3.1 텐서플로우(TensorFlow) ▶ 3.1.2 TensorFlow 설치

☑ 사전 준비

- 파이썬(버전 : 3.10.6) 설치
- vscode 설치, jupyter notebook 플러그인 설치

☑ 텐서플로우(TensorFlow) 설치

- 설치 명령어 : `pip install tensorflow`
- 텐서플로우 버전 확인 : `pip list → tensorflow 2.18.0`

☑ 기타 설치 라이브러리

- numpy 설치 명령어 : `pip install numpy`
- pandas 설치 명령어 : `pip install pandas`
- matplotlib 설치 명령어 : `pip install matplotlib`
- seaborn 설치 명령어 : `pip install seaborn`

3. 텐서플로우와 케라스

3.1 텐서플로우(TensorFlow) ▶ 3.1.3 기본구조

TensorFlow의 기본 구조

1. 데이터 준비 (Input)

- 데이터를 모델에 전달할 준비
- 예 : CSV 파일 읽기, 이미지 데이터 로드

2. 모델 정의 (Model)

- 신경망 모델의 구조를 정의
- 예 : 선형 회귀, 다층 퍼셉트론, CNN

3. 모델 학습 (Training)

- 손실 함수(Loss Function)를 최소화하도록 가중치 최적화
- 예 : 경사 하강법 (Gradient Descent)

4. 예측 및 평가 (Prediction & Evaluation)

- 학습된 모델을 사용해 새로운 데이터를 예측
- 성능을 평가

TensorFlow 예제

TensorFlow를 사용해 $y=2x+1$ 관계를 학습하는
선형 회귀 모델

```
import tensorflow as tf

# 데이터 준비 ( $y = 2x + 1$ )
X = tf.constant([[1], [2], [3], [4]], dtype=tf.float32) # 독립 변수
y = tf.constant([[3], [5], [7], [9]], dtype=tf.float32) # 종속 변수

# 모델 정의 ( $y = wx + b$ )
model = tf.keras.Sequential([
    # 입력 1개, 출력 1개
    tf.keras.layers.Dense(units=1, input_shape=[1])
])

# 모델 컴파일
model.compile(loss='mean_squared_error', optimizer='sgd')

# 모델 학습
model.fit(X, y, epochs=100)

# 예측
print(model.predict([[10]])) # x=10일 때 y 값 예측

# 모델 평가
test_loss = model.evaluate(X, y)
print('Test loss:', test_loss)
```

3. 텐서플로우와 케라스

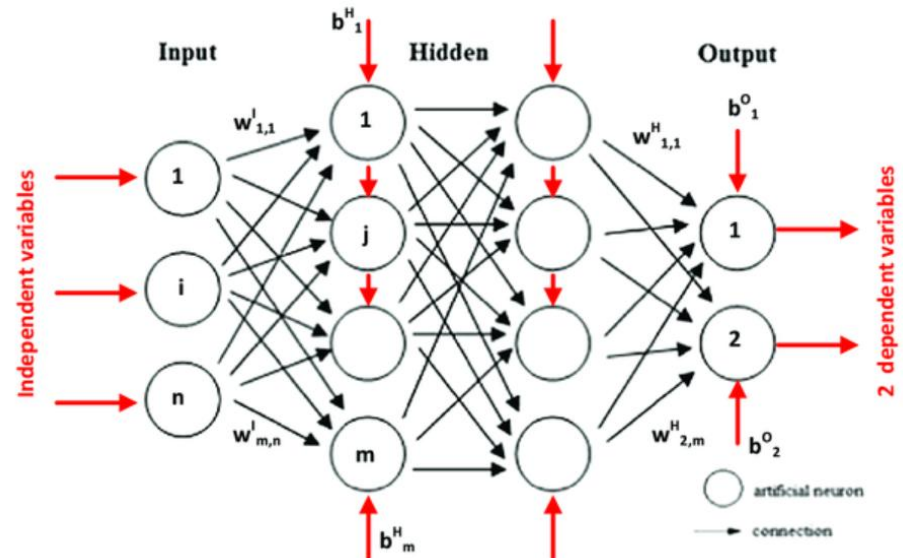
3.1 텐서플로우(TensorFlow) ▶ 3.1.3 기본구조

○ 용어 정리

- ☑ 순전파 : 입력 데이터가 신경망을 거쳐 출력이 생성
- ☑ 손실 계산 : 생성된 출력과 실제 정답 간의 차이를 손실 함수를 통해 계산
- ☑ 역전파 : 손실을 최소화하기 위해 각 뉴런의 가중치에 대한 기울기를 계산. 즉, 각 가중치가 손실에 얼마나 기여하는 지 계산
- ☑ 옵티마이저 : 역전파를 통해 얻은 기울기를 이용하여 가중치를 업데이트함. 옵티마이저는 어떤 방식으로 가중치를 업데이트 할지를 결정하는 알고리즘

○ 위의 용어에 대한 비유 예시

- ☑ 역전파 : 건물의 설계도를 보고 어떤 부분을 수정해야 건물이 더욱 튼튼해질지 계산하는 과정
- 예 : 손실에 대한 기여도 계산(함수 기울기 계산)
- ☑ 옵티마이저 : 계산된 수정 사항을 바탕으로 실제로 건물을 수정하는 작업
- 예 : 가중치를 조정(종류 : SGD, Adam)



3. 텐서플로우와 케라스

3.1 텐서플로우(TensorFlow) ▶ 3.1.4 텐서플로우 예제 (1/2)

- ☑ 데이터 준비 : x와 y 데이터를 numpy 배열로 정의하여 학습에 사용할 데이터를 준비
- ☑ 모델 정의 : $y = wx + b$ 형태의 간단한 선형 모델을 정의하고, 가중치와 편향을 학습 가능한 변수로 초기화
- ☑ 모델 학습 : 손실함수는 평균 제곱 오차(MSE)를 사용

```
import tensorflow as tf
import numpy as np

# 1. 데이터 준비
# x_data: 입력 데이터, y_data: 실제 값 (정답)
x_data = np.array([0, 1, 2, 3, 4], dtype=np.float32)
y_data = np.array([1, 3, 5, 7, 9], dtype=np.float32)

# 2. 모델 정의
# 가중치(w)와 편향(b)를 변수로 정의
w = tf.Variable(0.0) # 초기 가중치
b = tf.Variable(0.0) # 초기 편향

# 예측 함수 ( $y = wx + b$ )
def predict(x):
    return w * x + b

# 3. 손실 함수 정의
# 평균 제곱 오차 (Mean Squared Error, MSE) 사용
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
```

3. 텐서플로우와 케라스

3.1 텐서플로우(TensorFlow) ▶ 3.1.4 텐서플로우 예제 (2/2)

- ☑ 최적화 알고리즘 : 경사 하강법(SGD)을 사용해 가중치와 편향을 반복적으로 업데이트
- ☑ 학습 반복 : 100번의 학습 반복을 통해 손실이 줄어드는 과정을 확인
- ☑ 예측 및 평가 : 학습된 모델을 사용해 새로운 x 값에 대해 y 값을 예측 (ex. x=5일 때 y값 예측)

```
# 4. 최적화 알고리즘 선택
optimizer = tf.optimizers.SGD(learning_rate=0.01) # 확률적 경사 하강법

# 5. 학습 과정
for step in range(100): # 100번 반복
    with tf.GradientTape() as tape:
        y_pred = predict(x_data) # 예측값
        loss = loss_fn(y_data, y_pred) # 손실 계산

    # 가중치와 편향에 대한 경사도 계산 및 업데이트
    gradients = tape.gradient(loss, [w, b])
    optimizer.apply_gradients(zip(gradients, [w, b]))

    if step % 10 == 0: # 10번마다 로그 출력
        print(f"Step {step}, Loss: {loss.numpy()}, w: {w.numpy()}, b: {b.numpy()}")

# 6. 예측 및 결과 확인
print("Final Parameters:", f"w={w.numpy()}, b={b.numpy()}")
print("Prediction for x=5:", predict(5).numpy())
```

3. 텐서플로우와 케라스

3.2 케라스(keras)

☑ 케라스(keras)는 파이썬으로 작성된 오픈 소스 신경망 라이브러리

☑ 현재는 TensorFlow의 공식 고급 API로 통합되어 사용됨

특징

- 사용자 친화적 : 간단하고 일관된 인터페이스를 제공하여 사용하기 쉬움
- 모듈화 : 신경망 구성 요소를 자유롭게 조합하여 모델을 만들 수 있음
- 확장성 : 새로운 층(layers), 지표(metrics), 손실 함수를 쉽게 추가할 수 있음

장점

- 직관적인 API : 복잡한 신경망 모델을 쉽게 구현 가능
- 유연성 : 다양한 신경망 구조를 쉽게 만들 수 있음
- 풍부한 문서와 커뮤니티 : 학습 자료가 많고 활발한 커뮤니티

단점

- 텐서플로우 위에서 동작하므로 속도면에서 조금 느릴 수 있음

3. 텐서플로우와 케라스

3.2 케라스(keras) ▶ 3.2.1 기본구조

Keras의 기본 구조

1. 데이터 준비 (Input)

- 데이터를 입력 형태에 맞게 전처리
- 예 : 데이터를 numpy 배열로 전환, 정규화, 훈련/검증 데이터로 분리

2. 모델 정의 (Model)

- 신경망 모델의 구조를 정의
- Sequential API 또는 Functional API 사용

3. 모델 컴파일

- 손실 함수, 옵티마이저, 평가 지표를 설정

4. 모델 학습

- fit() 메서드를 사용해 데이터를 모델에 학습

5. 예측 및 평가

- 학습된 모델을 사용해 새로운 데이터에 대한 예측 수행
- 성능을 평가

Keras 예제

Keras를 사용해 $y=2x+1$ 관계를 학습하는 선형 회귀 모델

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 데이터 준비 ( $y = 2x + 1$ )
X = tf.constant([[1], [2], [3], [4]], dtype=tf.float32) # 독립 변수 (x)
y = tf.constant([[3], [5], [7], [9]], dtype=tf.float32) # 종속 변수 (y)

# 모델 정의 ( $y = wx + b$ )
model = Sequential([
    Dense(units=1, input_shape=[1]) # 입력 1개, 출력 1개
])

# 모델 컴파일
model.compile(loss='mean_squared_error', optimizer='sgd')

# 모델 학습
model.fit(X, y, epochs=1000)

# 예측
print(model.predict([[10]])) # x=10일 때 y 값 예측

# 모델 평가
test_loss = model.evaluate(X, y)
print('Test loss:', test_loss)
```


3. 텐서플로우와 케라스

3.2 케라스(keras) ▶ 3.2.1 케라스 예제 (1/2)

- ☑ 데이터 준비 : x와 y 데이터를 numpy 배열로 정의하여 학습에 사용할 데이터를 준비
- ☑ 모델 정의 : Sequential API를 사용해 신경망 구조를 Dense층을 사용해서 간단히 정의(입력층 1개, 출력층 1개)
- ☑ 모델 컴파일 : 손실 함수는 MSE(평균 제곱 오차) , 옵티마이저 SGD(확률적 경사 하강법) 사용

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 1. 데이터 준비
# 입력(x)과 출력(y) 데이터를 정의합니다.
x_data = np.array([0, 1, 2, 3, 4], dtype=np.float32) # 입력 데이터
y_data = np.array([1, 3, 5, 7, 9], dtype=np.float32) # 정답 데이터

# 2. 모델 정의
# Sequential API를 사용해 신경망을 정의합니다.
model = Sequential([
    Dense(1, input_dim=1) # 입력 1개, 출력 1개 (선형 회귀)
])

# 3. 모델 컴파일
# 손실 함수와 옵티마이저를 설정합니다.
model.compile(optimizer='sgd', loss='mse')
```

3. 텐서플로우와 케라스

3.2 케라스(keras) ▶ 3.2.2 케라스 예제 (2/2)

- ☑ 모델 학습 : fit() 메서드로 데이터를 모델에 학습시키는데, 데이터를 50번 반복 학습(epochs=50)
- ☑ 예측 및 평가 : 학습된 모델을 사용해 새로운 입력 x=5에 대한 y 값을 예측

```
# 4. 모델 학습
# fit() 메서드로 데이터를 학습시킵니다.
model.fit(x_data, y_data, epochs=50, verbose=0) # 50번 반복 학습

# 5. 예측 및 평가
# 새로운 데이터로 예측 수행
x_new = np.array([5], dtype=np.float32)
y_new = model.predict(x_new)

print(f"x = 5일 때, 예측된 y = {y_new[0][0]:.2f}")
```

4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.1 합성곱 신경망 정의 및 특징 (1/3)

- ☑ 이미지, 비디오, 음성 신호와 같은 데이터에서 특징을 자동으로 추출하고 분석하는 데 최적화된 딥러닝 모델
- ☑ 주로 컴퓨터 비전 분야에서 사용되며, 이미지를 처리하고 패턴(엣지, 색모양 등)을 학습하는 데 뛰어남

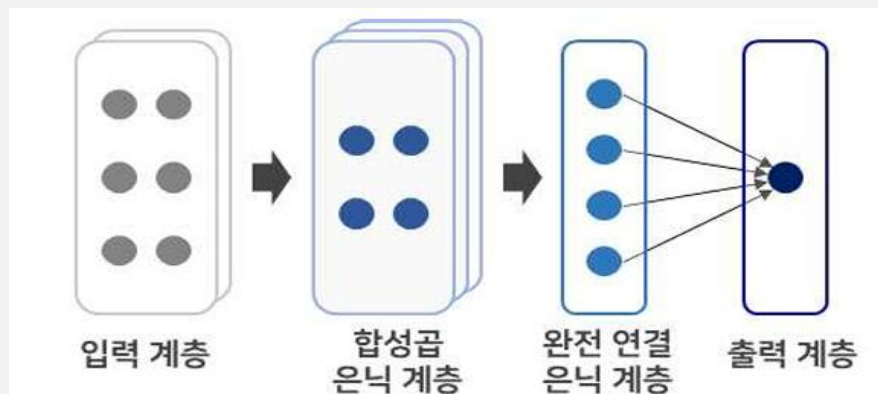
활성화 함수 특징

- 대상에 대한 특징을 인식하는 방식을 모델링
- 이미지 데이터를 다양한 방법으로 필터링하여 학습
- 이미지 학습을 위한 효율적인 알고리즘



CNN 구조

- 주로 입력 부근 계층들을 합성곱 계층으로 구성하고 출력 부근 층들을 완전 연결 계층으로 구성



4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.1 합성곱 신경망 정의 및 특징 (2/3)

- ☑ 합성곱 계층 (convolution Layer)
 - 입력 데이터에 필터(커널)를 적용하여 특징 맵(feature map)을 생성
- ☑ 활성화 함수 (Activation Function)
 - 주로 ReLU를 사용하여 비선형성을 추가
- ☑ 풀링 계층 (Pooling Layer)
 - 데이터 크기를 줄이고 학습속도와 성능을 향상
- ☑ 완전 연결 계층 (Fully Connected Layer)
 - 최종적으로 클래스 분류 또는 회귀 문제를 해결

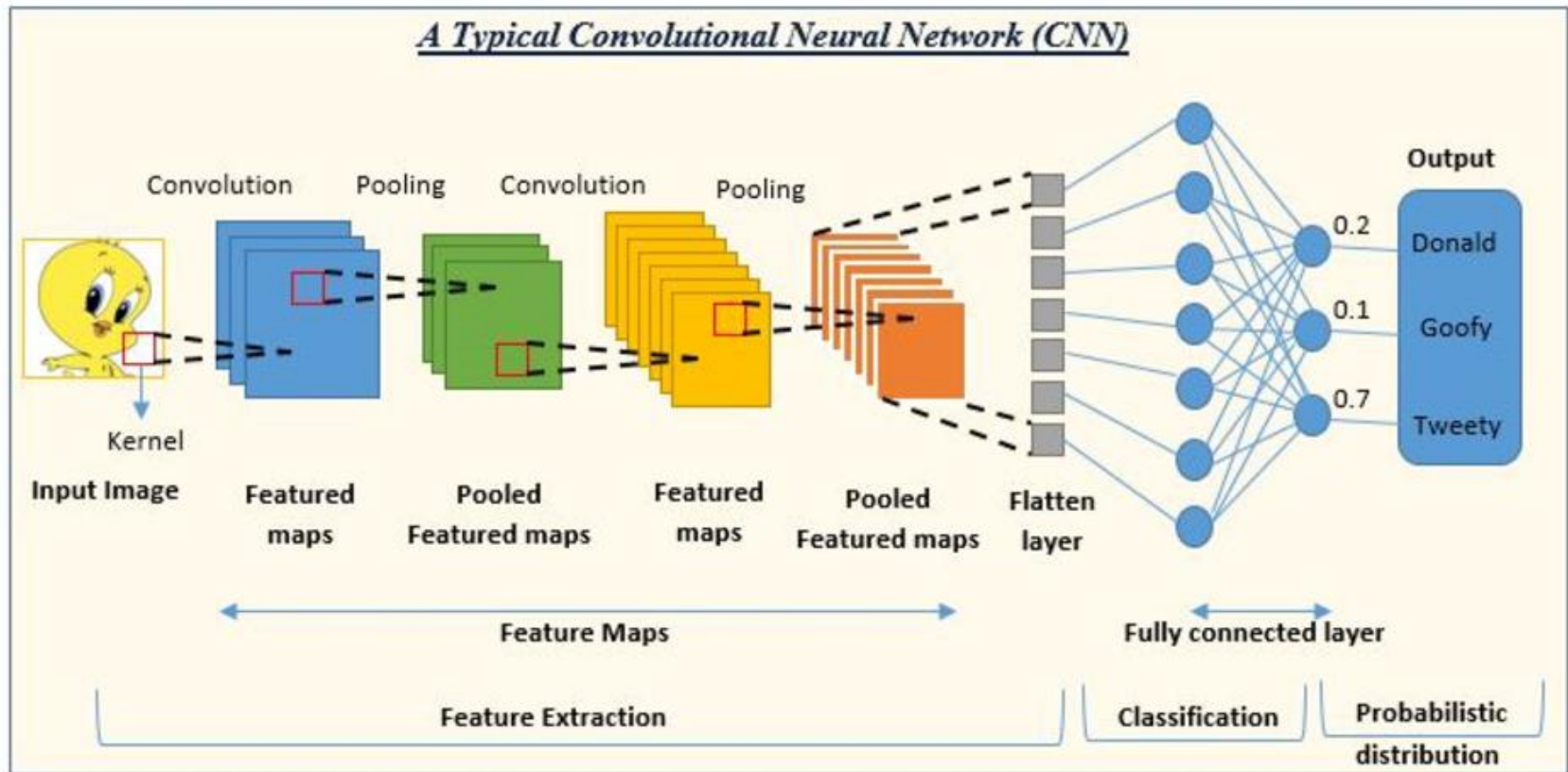
활용 예시

- 이미지 분류 : 다양한 이미지를 통한 고양이와 개를 구분하는 모델
- 객체 탐지 : 사진에서 자동차, 사람 등을 찾아내는 모델
- 의료 영상 분석 : MRI나 CT 스캔 데이터를 분석하여 질병을 탐지

4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.1 합성곱 신경망 정의 및 특징 (3/3)

☑ 전형적인 CNN 구조 예시

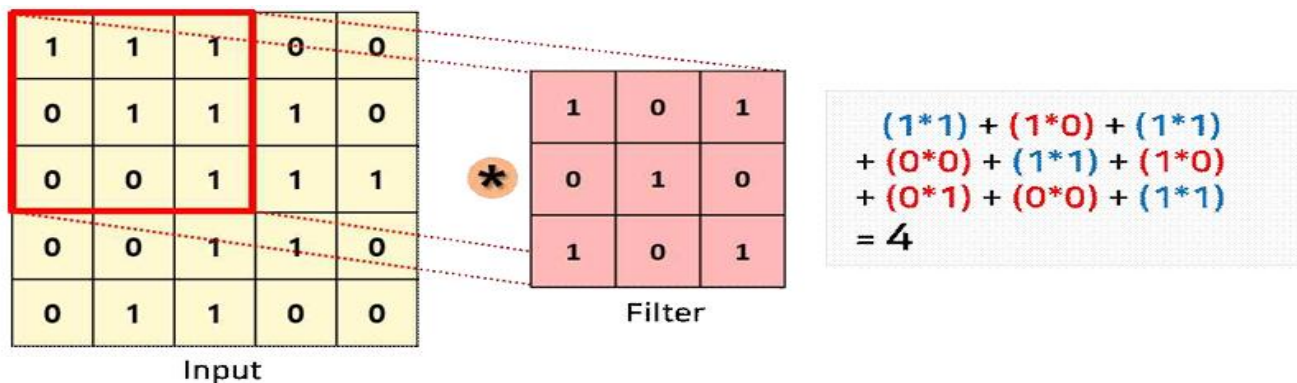


4. 합성곱 신경망(CNN, Convolutional Neural Network)

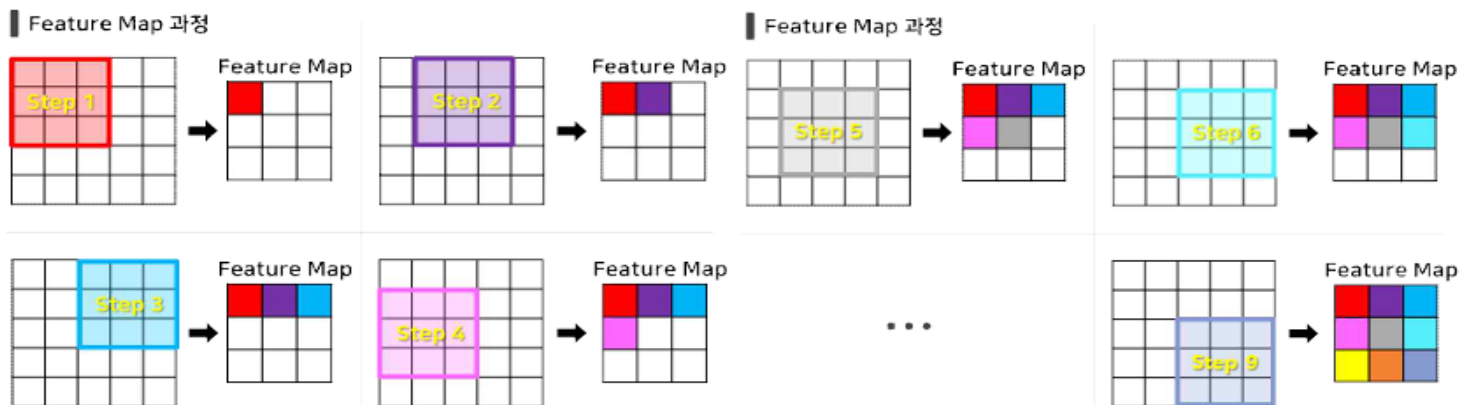
4.2 합성곱 은닉 계층 ▶ 합성곱 Layer

○ 합성곱 (Convolution) 연산

- ☑ 입력 데이터에 필터를 적용해 일정 간격으로 이동해가며 입력 데이터에 적용



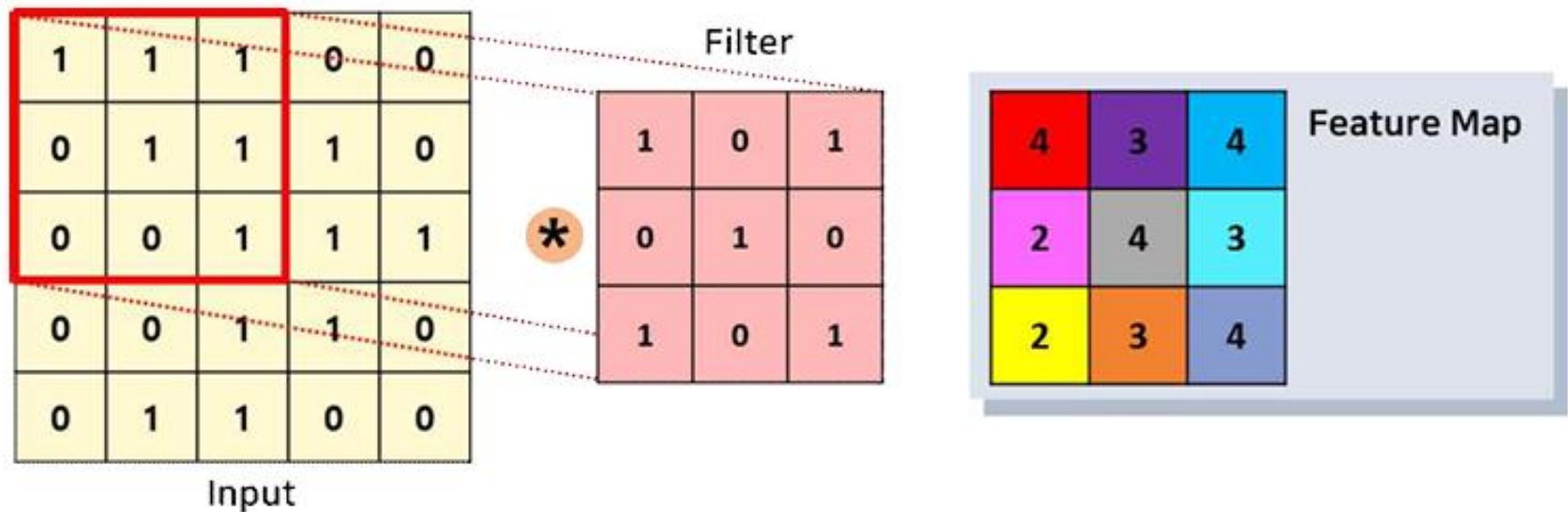
☑ 합성곱 연산 과정



4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ 합성곱 Layer

☑ 합성곱 연산 결과

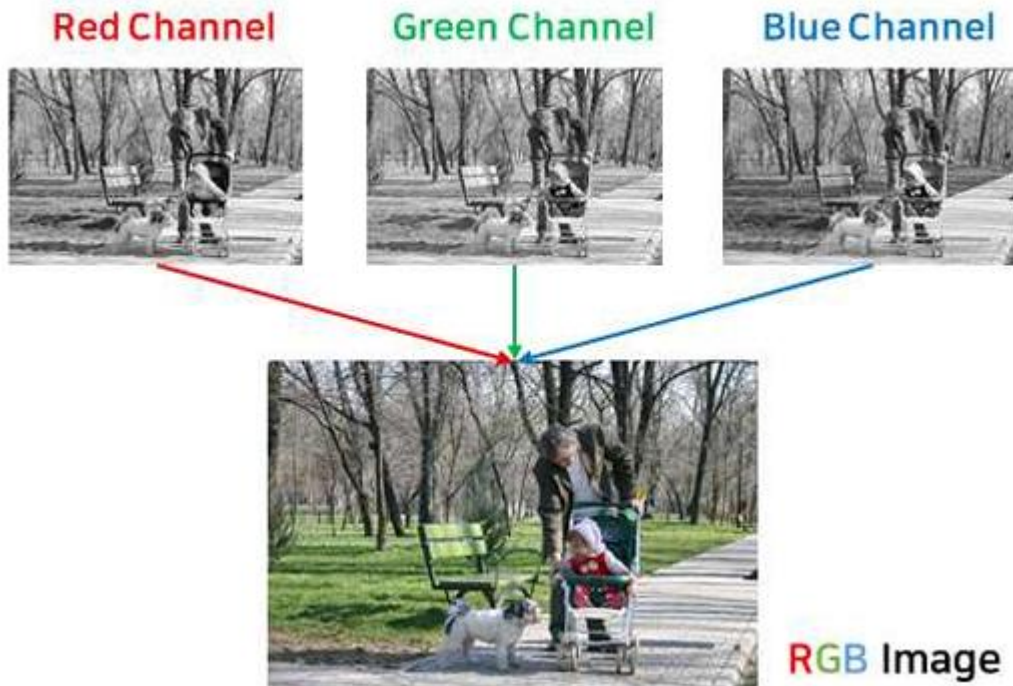


4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ 합성곱 Layer

○ 채널(Channel)

- ☑ 컬러 이미지는 각 픽셀을 RGB 3개의 실수로 표현한 3차원 데이터로 3개의 채널(Channel)로 구성

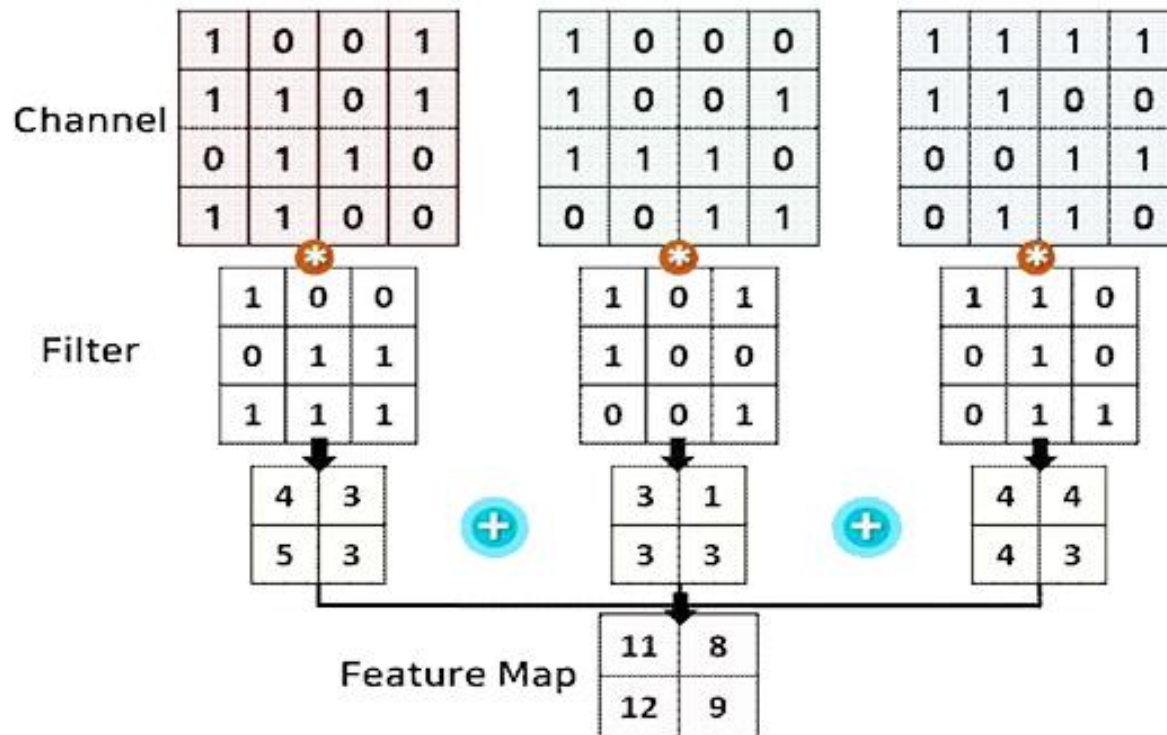


4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ 합성곱 Layer

○ 차원 데이터 합성곱

☑ 주어진 채널 수만큼 각각의 다른 필터를 적용하여 연산



4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ 합성곱 Layer

○ 패딩(Padding)

- ☑ Convolution Filter → 출력 이미지의 사이즈가 작아지게 됨
- ☑ 패딩(Padding) → 출력 이미지 사이즈를 그대로 유지 가능

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

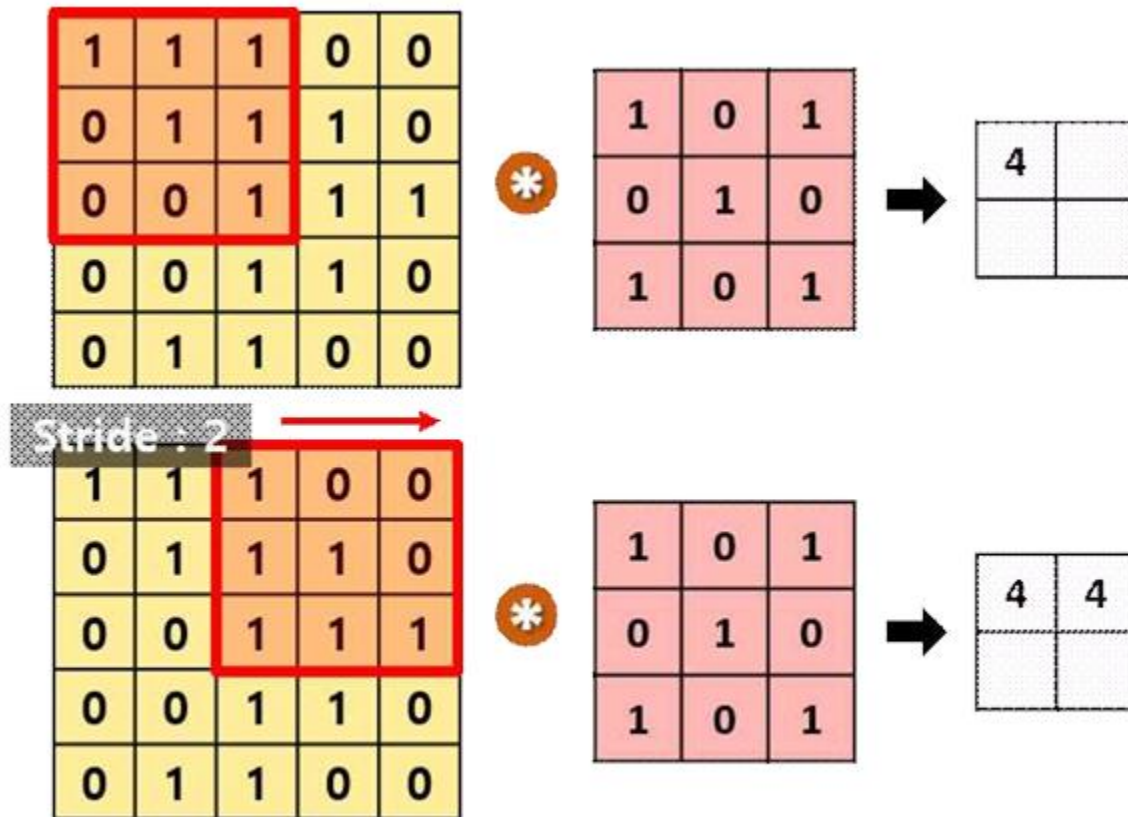
Padding 값을 1으로 준 결과

4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ 합성곱 Layer

○ 스트라이드(Stride)

- ☑ 입력 데이터에 필터를 적용할 때 이동할 간격을 조정하는 것



4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ Pooling Layer

○ Pooling

- ☑ sub-sampling을 이용해 특징맵(feature-map)의 크기를 줄이고, 위치나 이동에 좀 더 강인한 성질을 갖는 특징 추출이 가능
- ☑ Max Pooling : 각 Pixel에서 최대값을 뽑아내는 과정

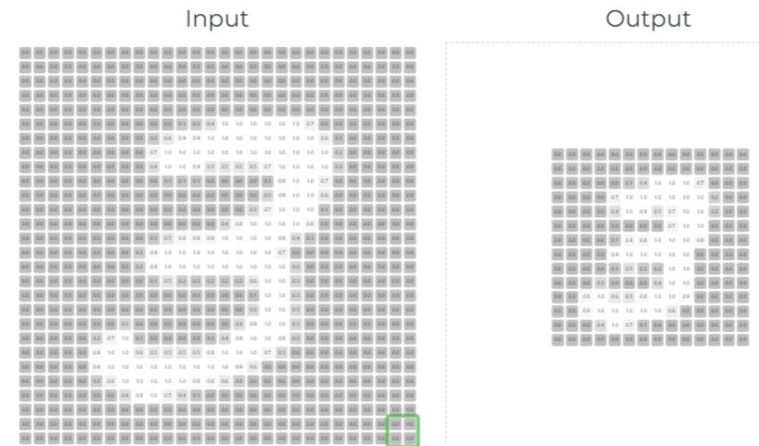
단일 채널 이미지

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4



6	8
3	4

Max Pooling 이후
줄어든 이미지



4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ Pooling Layer

☑ Average Pooling : 각 Pixel에서 평균값을 뽑아내는 과정

☑ 평탄화: 2/3차원의 행렬 을1차원 Vector로 변환하는 과정

단일 채널 이미지

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4



3.25	5.25
2	2

Average Pooling 이후
줄어든 이미지

1	2	3
4	5	6
7	8	9

Pooled Feature Map



Flattening

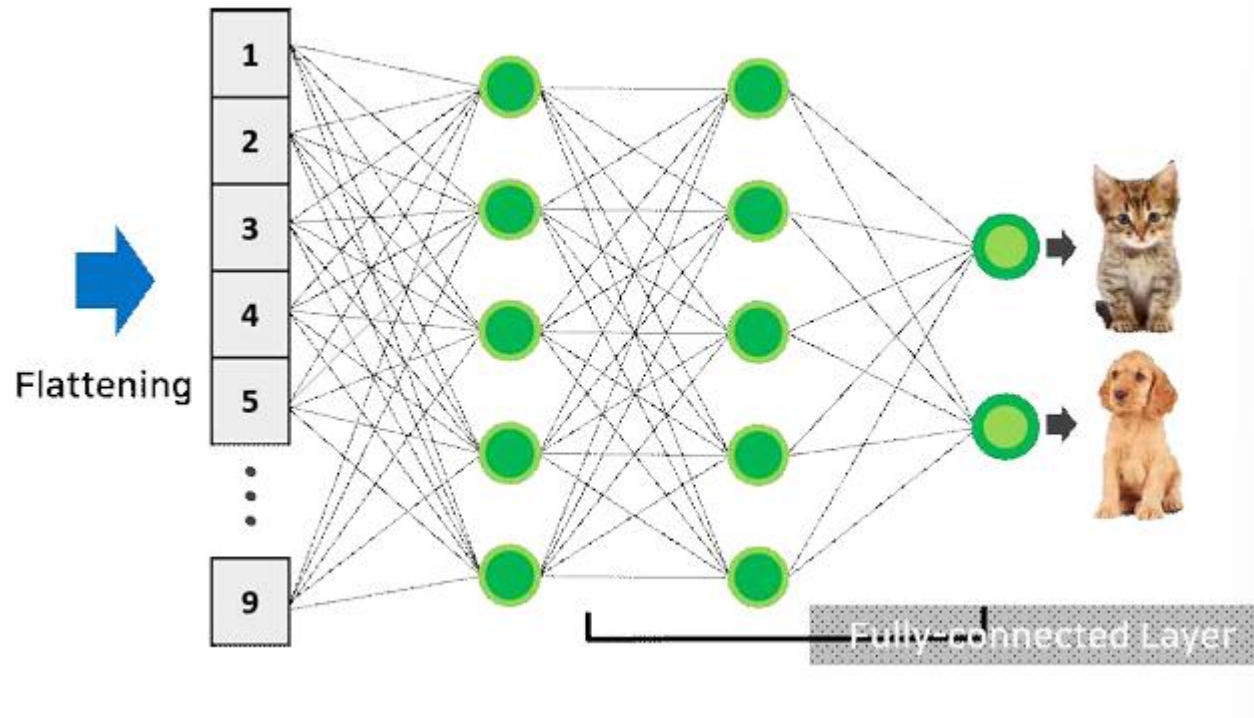
1
2
3
4
5
6
7
8
9

4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.2 합성곱 은닉 계층 ▶ 완전 연결 계층

○ 완전 연결 계층

- ☑ 이전 계층의 모든 노드가 다음 계층의 모든 노드에 연결된 계층



4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.3 MNIST 필기체를 구분하는 분류 CNN 구조

MNIST CNN 기본 구조

1. 데이터 준비 (Input)

- MNIST 데이터셋 로드 및 전처리

2. 모델 정의 (Model)

- CNN 모델 설계 (Conv2D, MaxPooling, Flatten, Dense 계층)

3. 모델 컴파일

- 손실 함수, 옵티마이저, 평가 지표를 설정

4. 모델 학습

- 학습 데이터로 모델 훈련

5. 예측 및 평가

- 학습된 모델을 사용해 임의의 샘플 이미지에 대한 예측 수행 및 성능평가

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# MNIST 데이터 로딩
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 데이터 전처리
x_train = x_train / 255.0
x_test = x_test / 255.0

# 모델 정의
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# 모델 컴파일
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 모델 학습
model.fit(x_train, y_train, epochs=5)

# 모델 평가
test_loss, test_acc = model.evaluate(x_test, y_test)
print('Test accuracy:', test_acc)
```


4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.3 MNIST 필기체를 구분하는 분류 CNN 구조 ▶ 4.3.1 CNN 예제 (1/4)

☑ 데이터 로드 및 전처리

- MNIST 데이터셋은 tensorflow.keras.datasets에서 제공
- reshape로 데이터를 CNN 모델에 맞게 채널 추가 및 정규화(픽셀 값 0~1)
- 레이블은 One-Hot Encoding으로 분류 문제에 적합하게 변환
- cf. One-Hot Encoding : 범주형 데이터를 수치형 데이터로 변환하는 방법(특정 클래스 1, 나머지는 모두 0으로 표현)

```
# 1. 필요한 라이브러리 임포트
import tensorflow as tf # 딥러닝 라이브러리
from tensorflow.keras import layers, models # 신경망 계층 및 모델 설계
from tensorflow.keras.datasets import mnist # MNIST 데이터셋
import matplotlib.pyplot as plt # 시각화 도구

# 2. 데이터 로드 및 전처리
# MNIST 데이터셋은 이미 나뉘어 제공 (학습/테스트 데이터)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# MNIST 이미지는 28x28 크기, 채널 정보 추가 (흑백 이미지는 채널 1개)
x_train = x_train.reshape((x_train.shape[0], 28, 28, 1)).astype('float32') / 255.0 # 0~1로 정규화
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1)).astype('float32') / 255.0

# 레이블을 One-Hot Encoding (숫자 0~9를 벡터로 변환)
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```


4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.3 MNIST 필기체를 구분하는 분류 CNN 구조 ▶ 4.3.1 CNN 예제 (2/4)

☑ 모델 정의

- Conv2D : 필터를 적용해 입력 데이터의 특징을 추출
- MaxPooling2D : 공간적 크기를 줄여 계산량 감소 및 특징 강화
- Flatten : 2차원 데이터를 1차원 벡터로 변환하여 Dense 계층에 연결
- Dense : Fully Connected Layer로 분류 결과를 도출

3. CNN 모델 정의

```
model = models.Sequential() # 순차적 모델 선언
```

첫 번째 합성곱 계층: 필터 크기 3x3, 필터 수 32, 활성화 함수 ReLU

```
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
```

풀링 계층: 2x2 크기로 다운샘플링

```
model.add(layers.MaxPooling2D((2, 2)))
```

두 번째 합성곱 계층: 필터 수 64

```
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

```
model.add(layers.MaxPooling2D((2, 2)))
```

세 번째 합성곱 계층: 필터 수 64

```
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

Fully Connected Layer를 연결하기 위해 Flatten 처리

```
model.add(layers.Flatten())
```

Fully Connected Layer 1: 뉴런 수 64, 활성화 함수 ReLU

```
model.add(layers.Dense(64, activation='relu'))
```

출력층: 뉴런 수 10(클래스 수), 활성화 함수 Softmax

```
model.add(layers.Dense(10, activation='softmax'))
```

모델 요약 출력 (구조 확인용)

```
model.summary()
```

4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.3 MNIST 필기체를 구분하는 분류 CNN 구조 ▶ 4.3.1 CNN 예제 (3/4)

- ☑ 모델 컴파일 : `compile` → 학습에 사용될 최적화 알고리즘, 손실함수, 평가지표 등을 설정
- ☑ 모델 학습 : `fit` → 학습 과정 수행(`epoch`와 `batch_size` 설정)
- ☑ 모델 및 평가 : `evaluate` → 테스트 데이터로 모델 성능 평가

```
# 4. 모델 컴파일
model.compile(optimizer='adam',          # 최적화 알고리즘(모델이 최적의 가중치를 찾도록 안내 역할)
               loss='categorical_crossentropy', # 다중 클래스 분류 손실 함수
               metrics=['accuracy'])        # 평가 지표: 정확도

# 5. 모델 학습
history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.2)

# 6. 모델 평가
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"테스트 정확도: {test_acc:.4f}")
```

4. 합성곱 신경망(CNN, Convolutional Neural Network)

4.3 MNIST 필기체를 구분하는 분류 CNN 구조 ▶ 4.3.1 CNN 예제 (4/4)

- ☑ 결과 확인 : predict → 임의의 샘플 데이터를 입력해 모델의 예측 확인
- ☑ 결과 출력 : 시각화를 통해 실제 값과 예측 값을 비교

```
# 7. 결과 확인: 테스트 데이터에서 임의의 이미지 예측
import numpy as np

# 테스트 데이터에서 첫 번째 샘플을 가져옴
sample_image = x_test[0]
sample_label = np.argmax(y_test[0])

# 모델로 예측 수행
predicted_label = np.argmax(model.predict(sample_image.reshape(1, 28, 28, 1)))

# 결과 출력
plt.imshow(sample_image.reshape(28, 28), cmap='gray')
plt.title(f"실제 값: {sample_label}, 예측 값: {predicted_label}")
plt.show()
```

5. RNN과 LSTM

5.1 순환신경망(RNN) ▶ 5.1.1 RNN의 기본 개념

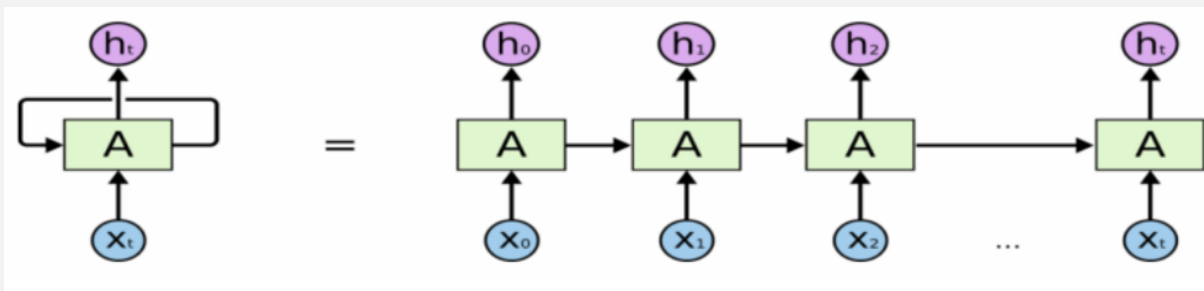
- ☑ RNN은 **순차적인 데이터를 처리**하기 위해 특별히 설계된 인공지능망
- ☑ 마치 인간의 기억처럼 **이전의 정보를 기억하고 현재의 정보와 연결**하여 처리할 수 있는 인공지능망(ex. 인간의 대화방식)

RNN 특징

- 텍스트, 음성, 시계열 데이터와 같이 순서가 중요한 데이터를 처리하는데 탁월
- 이전 입력 정보를 기억하고 현재 처리에 반영하며, 시간에 따라 정보를 전달하고 업데이트
- 같은 네트워크 가중치를 시간에 따라 반복적으로 적용, 이를 통해 입력 시퀀스 길이에 상관없이 처리

RNN 작동원리

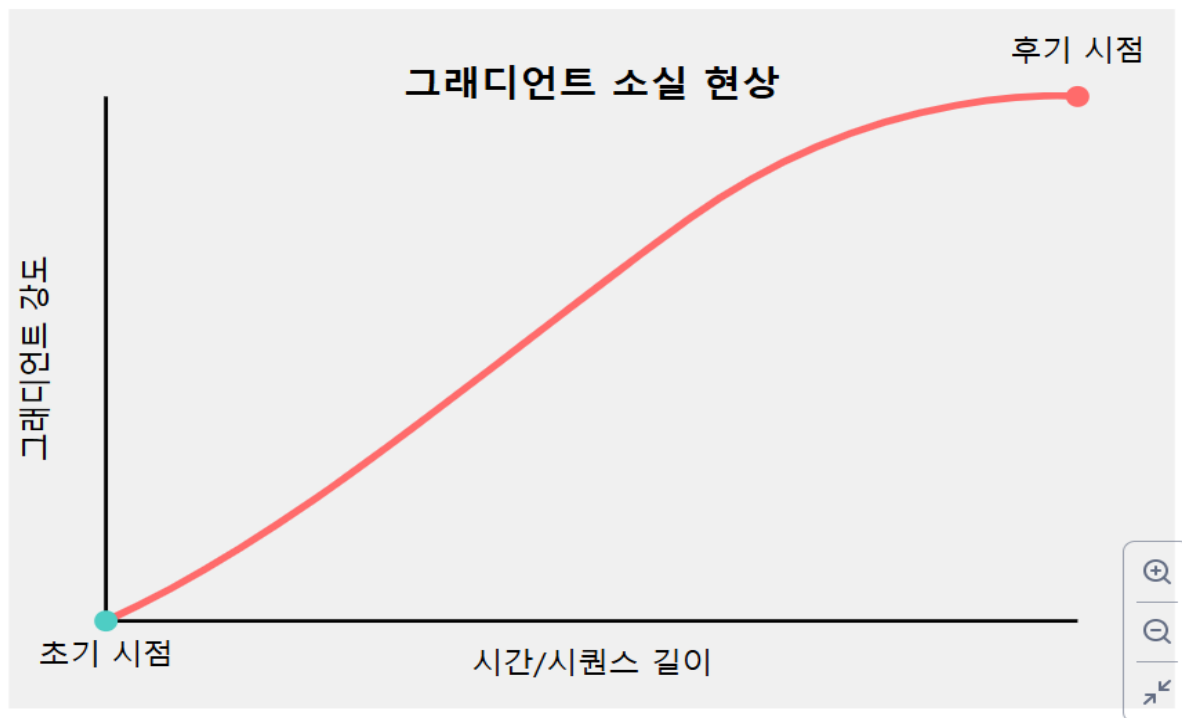
- 아래 그림에서 X 는 입력값이고, A 가 RNN 활성화 함수이며, h_t 는 결과 값을 의미
- 내부의 유닛 사이의 연결이 Directed Cycle(다이렉트 사이클) 형성
- 자기 자신을 가리키는 순환 가중치(Reccurent Weight)를 포함하는 네트워크로 구성



5. RNN과 LSTM

5.1 순환신경망(RNN) ▶ 5.1.2 RNN의 한계

- ☑ RNN은 **입력데이터가 길어지면 기울기 소실 문제 즉, 장기 의존성 문제**가 발생하여 이전의 정보를 제대로 기억하지 못함
- ☑ 기울기 소실(**gradient vanishing**)이란 신경망 학습 과정에서 초기 시점의 정보가 점점 희미해지는 현상을 의미함
- ☑ 각 시점의 그래디언트가 곱해지며 전파되는데, 연속적인 곱셈으로 인해 기울기 값이 기하급수적으로 작아짐
- ☑ 실제 영향과 예시 : **[기계번역] 문장의 첫 부분 컨텍스트 완전 상실, [음성인식] 초기 발음 정보 소실**



5. RNN과 LSTM

5.2 LSTM ▶ 5.2.1 LSTM의 기본 개념

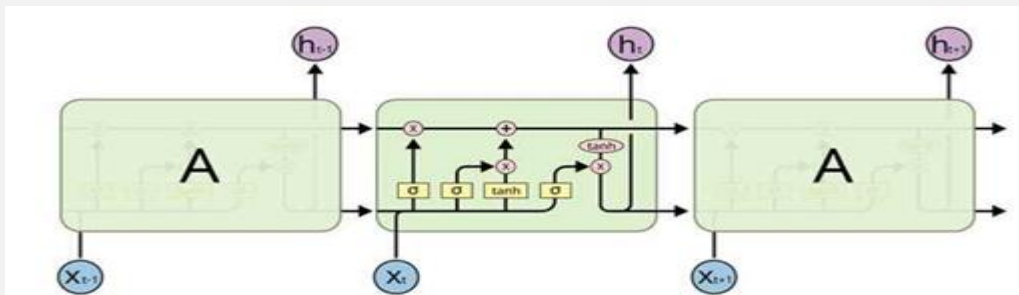
- ☑ LSTM은 RNN의 장기 의존성 문제를 해결하기 위해 개발된 특별한 유형의 순환 신경망
- ☑ 마치 지능적인 메모리 관리자와 같이 중요한 정보는 오래 기억, 불필요한 정보는 빠르게 잊는 능력을 가진 신경망

LSTM 특징

- RNN의 장기 의존성 문제를 해결하여 긴 시퀀스의 데이터에서도 중요한 정보를 오래 기억할 수 있음
- 정보의 선택적 기억과 망각이 가능
- 망각게이트, 입력 게이트, 출력 게이트의 세 개의 게이트로 정보를 정교하게 제어

LSTM 작동원리

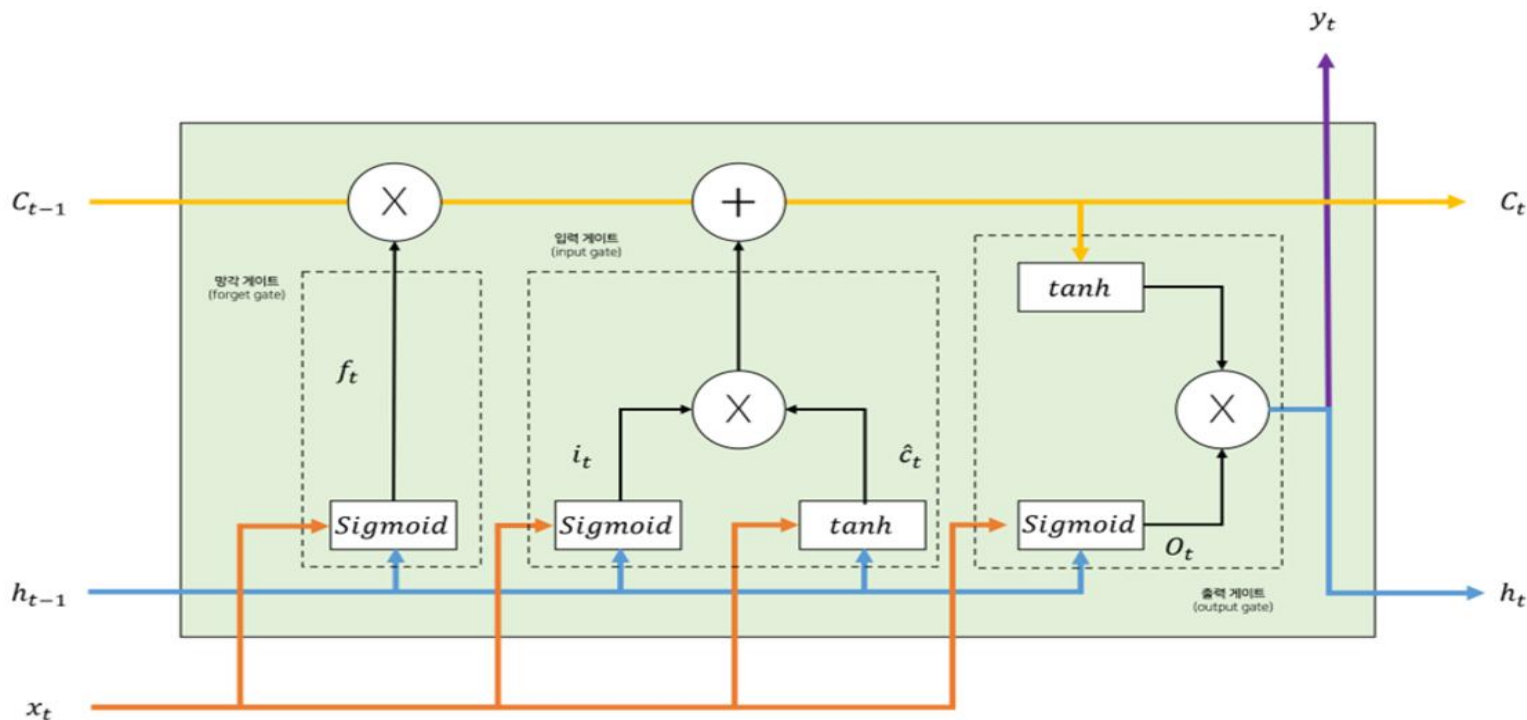
- 아래 그림에서 X 는 입력값이고, A 가 RNN 활성화 함수이며, h_t 는 결과 값을 의미
- 가운데에 보이는 메모리 셀 안에서 상태 정보를 변경하는 게이트를 가지고 있음



5. RNN과 LSTM

5.2 LSTM ▶ 5.2.2 게이트별 상세 메커니즘

- ☑ 망각 게이트 : 이전의 어떤 정보를 버릴 지 결정. 0과 1사이의 값으로 정보 보존 정도 결정. (ex. 문맥에서 관련없는 정보 제거)
- ☑ 입력 게이트 : 새로운 정보를 추가할 지 결정. 현재 입력 중 어떤 부분을 기억할 지 선택. (ex. 대화에서 중요키워드 선택/기억)
- ☑ 출력 게이트 : 현재 셀 상태를 기반으로 출력 결정. 어떤 부분을 다음 단계로 전달할 지 선택. (ex. 문장의 핵심정보 추출/전달)



* 비유로 이해하는 LSTM : 셀상태(메모장), 망각게이트(메모 지우기), 입력게이트(메모 추가), 출력게이트(메모장의 내용 중 필요부분 공유)

5. RNN과 LSTM

5.2 LSTM ▶ 5.2.3 LSTM 시계열 구조(1/4)

Feature (특징)

- 정의 : 시계열 데이터에서 각 데이터 포인트를 구성하는 요소
- 예시 : 주식 데이터의 경우, feature는 시가, 고가, 저가, 종가 // 기온데이터는 기온, 습도
- 역할 : LSTM 모델은 이러한 feature들을 입력으로 받아 시간에 따른 변화 패턴을 학습

Sequence Length (시퀀스 길이)

- 정의 : LSTM 모델에 입력되는 시퀀스(순서열)의 길이
- 예시 : 만약 5일 동안의 주가 데이터를 이용하여 6일차 주가를 예측한다면 sequence length = 5
- 역할 : sequence length는 모델이 과거 데이터를 얼마나 많이 참고하여 예측할 지 결정
 - 짧은 sequence length : 최근 데이터에 더 큰 영향을 받아 단기적 패턴을 잘 학습
 - 긴 sequence length : 장기적인 패턴을 학습할 수 있지만, 너무 길면 계산량 증대 및 과적합문제

Step (시간간격)

- 정의 : 시퀀스를 구성하는 각 데이터 포인트 사이의 시간 간격
- 예시 : 주가 데이터의 경우, step은 하루, 일주일 등이 될 수 있음
- 역할 : step은 시계열 데이터의 주기성을 반영하는 중요한 요소
 - 짧은 step : 고빈도 데이터의 경우, 짧은 step을 사용하여 빠르게 변화하는 패턴을 포착
 - 긴 step : 저빈도의 데이터의 경우, 긴 step을 사용하여 장기적인 추세를 파악

5. RNN과 LSTM

5.2 LSTM ▶ 5.2.3 LSTM 시계열 구조(2/4)

○ 주가예측 시퀀스 데이터 예시

- ☑ feature : 시가, 고가, 저가, 종가
- ☑ sequence length : 2 (2일 동안의 데이터를 이용하여 3일차 주가를 예측)
- ☑ step : 1일

날짜	시가	고가	저가	종가
2023-11-20	100	110	95	105
2023-11-21	105	115	100	112
2023-11-22	112	120	110	118

○ LSTM 모델 학습 시 고려사항

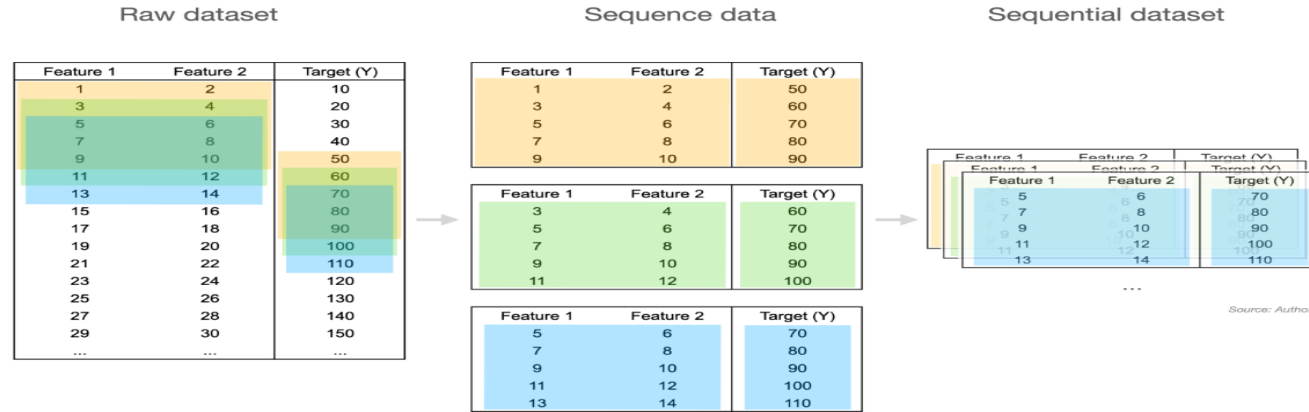
- ☑ 데이터 전처리 : 데이터의 스케일링, 정규화 등을 통해 모델 학습 효율을 높일 수 있음
- ☑ 하이퍼파라미터 튜닝 : sequence length, step, LSTM 유닛 수, 학습률 등을 조절하여 최적의 모델을 찾음
- ☑ 과적합 방지 : dropout(드롭아웃), L1/L2 정규화 등을 사용하여 과적합 문제 해결

- 과적합 : 모델이 학습 데이터에 너무 완벽하게 맞춰져서 새로운 데이터에 대해 일반화 능력이 떨어지는 현상
- **dropout** : 신경망 학습 시, **각 뉴런을 일정 확률로 비활성화시켜 모델의 복잡도를 줄이는 방법**
- L1/L2 정규화 : 모델의 파라미터(가중치)에 제약을 가하여 불필요한 복잡성을 줄이는 것 (**L1 정규화** : 파라미터 절대값 합을 손실함수에 추가, **많은 파라미터를 0으로 만들어 특징 선택 효과**, **L2 정규화** : 파라미터의 제곱의 합을 손실 함수에 추가, 모든 **파라미터를 작게 만들어 과적합 방지**)

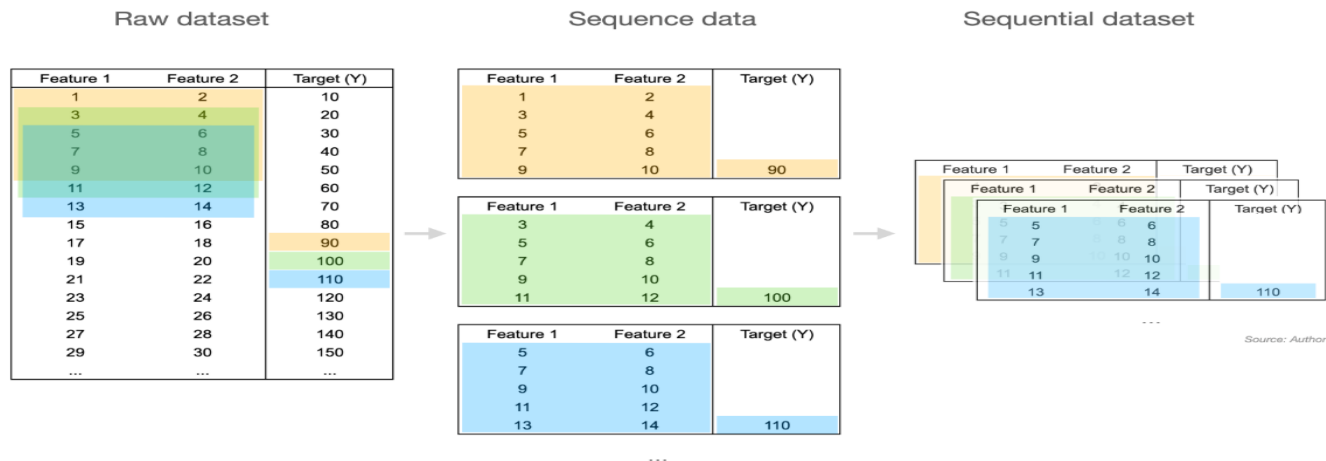
5. RNN과 LSTM

5.2 LSTM ▶ 5.2.3 LSTM 시계열 구조(3/4)

○ 시계열 데이터셋 예시 (sequence length = 5, step = 5 인 경우)



○ 시계열 데이터셋을 통한 예측할 값이 한 개인 경우(single output = True)



5. RNN과 LSTM

5.2 LSTM ▶ 5.2.3 LSTM 시계열 구조(4/4)

○ 시계열 데이터셋 예시 (sequence length = 3, step = 1 인 경우)

시간	온도 (°C)	습도 (%)	PM10 ($\mu\text{g}/\text{m}^3$)
1	25	60	45
2	26	62	50
3	27	65	55
4	28	63	60
5	29	61	65
6	30	58	70
7	31	55	75

- 첫 번째 **Sequence**:

- X:
[25, 60, 45], # 시간 1
[26, 62, 50], # 시간 2
[27, 65, 55] # 시간 3

- y: [60] # 시간 4의 PM10 값

- 두 번째 **Sequence**:

- X:
[26, 62, 50], # 시간 2
[27, 65, 55], # 시간 3
[28, 63, 60] # 시간 4

- y: [65] # 시간 5의 PM10 값

5. RNN과 LSTM

5.3 RNN과 LSTM 비교

구분	RNN	LSTM
구조	단순 순환 구조로 이전 상태와 입력을 결합하여 현재 상태를 계산	RNN 구조에 셀상태와 게이트 구조를 추가하여 정보 흐름을 조절
핵심 특징	- 시퀀셜 데이터의 시간적 의존성을 처리 - 과거 정보를 기반으로 다음 상태 계산	- 입력 게이트, 출력 게이트, 망각 게이트로 정보 흐름 제어 - 장기 의존성 문제 해결 가능
장점	- 간단한 구조와 구현 용이성 - 짧은 시퀀스 데이터 처리에 적합	- 장기 의존성 처리가 가능 - 정보 손실 방지 및 중요한 정보 유지
단점	- 장기 의존성 문제 발생 가능 - 정보 손실과 계산 비효율성	- 구조적으로 복잡하여 계산 비용 증가 - 모델 학습 속도가 느림
사용사례	- 간단한 언어 모델링 - 시계열 데이터 분석(주가 예측 등) - 음성 인식(기본 수준)	- 자연어 처리(NLP) : 번역, 텍스트 생성 - 음성 인식 - 시계열 예측, 비디오 분석 등
장기 의존성 처리	어려움 : 이전 상태가 멀어질수록 정보가 희석되거나 소실	가능 : 망각 게이트를 통해 불필요한 정보 제거, 셀 상태로 정보 유지
계산 복잡도	비교적 낮음	상대적으로 높음
기타사항	- 간단한 문제를 해결할 때 적합	- CNN, Transformer와 결합하여 활용 가능 - LSTM 단점 보완을 위해 GRU 선택 가능

5. RNN과 LSTM

5.4 GRU ▶ 5.4.1 GRU의 기본 개념

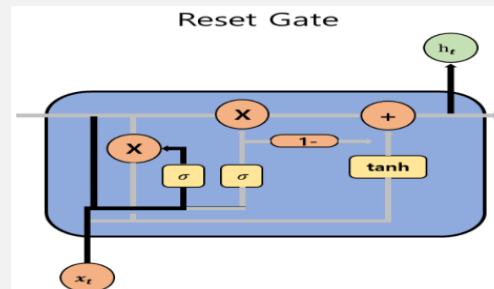
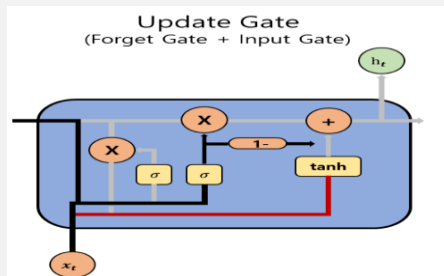
- ☑ GRU(Gated Recurrent Unit)는 LSTM의 개선된 버전
- ☑ LSTM과 유사한 구조를 가지면서도 **더 적은 파라미터를 사용해 계산량을 줄이고 성능을 유지**할 수 있도록 설계됨

GRU 특징

- LSTM은 3개의 게이트(입력, 망각, 출력 게이트)를 사용하지만, **GRU는 2개의 게이트(업데이트, 리셋 게이트)만 사용하여 구조가 더 단순함**
- 파라미터 수 감소 → 학습 속도 향상 (연산량이 줄어들고 훈련 속도가 LSTM보다 빠름)
- 장기 의존성 해결 : LSTM처럼 기울기 소실 문제 완화, 장기적인 정보도 잘 유지

LSTM 작동원리

- 업데이트 게이트 : 새로운 정보와 기존 정보를 얼마나 유지할 지 결정하는 역할
- 리셋 게이트 : 이전 상태의 정보를 얼마나 잊을지를 결정



5. RNN과 LSTM

5.4 GRU ▶ 5.4.2 LSTM과 GRU 비교

- ☑ GRU는 LSTM보다 가벼운 모델이지만, 성능은 거의 유사한 경우가 많아 계산 자원이 **제한된 환경에서는 GRU가 유리**
- ☑ 반면, **더 복잡한 패턴을 학습해야 하는 경우 LSTM이 더 적합**할 수 있음

구분	LSTM	GRU
게이트 개수	3개 (입력, 망각, 출력)	2개 (업데이트, 리셋)
셀 상태	있음 (셀 상태와 은닉 상태 분리)	없음 (은닉 상태만 존재)
파라미터 수	상대적으로 많음	상대적으로 적음
계산량	많음 → 학습 속도 느림	적음 → 학습 속도 빠름
장기의존성 처리	더 강력한 장기 의존성 유지	가능하지만 LSTM보다 덜 강력할 수 있음
일반적인 성능	일부 복잡한 작업에서는 더 나은 성능	대부분의 경우 LSTM과 비슷하거나 더 나옴
사용 예시	챗봇, 자연어 처리, 복잡한 시퀀스 데이터	음성 인식, 기계 번역, 시계열 예측

5. RNN과 LSTM

5.5 시계열 예측을 위한 RNN 및 LSTM 구조

RNN 및 LSTM 기본 구조

1. 데이터 준비 (Input)

- 데이터를 전처리하고, 시퀀스 형태로 변환
- 데이터셋을 훈련용과 테스트용으로 분리

2. 모델 정의 (Model)

- RNN 또는 LSTM 계층을 포함하는 Sequential 모델 정의
- 시퀀스 데이터 형태를 처리할 수 있도록 입력크기 설정

3. 모델 컴파일

- 손실 함수, 최저고하 알고리즘(optimizer)을 지정
- 성능 평가를 위한 지표(metrics)를 추가

4. 모델 학습

- 모델에 데이터를 입력하고 학습(epoch)과정을 진행
- 학습 데이터를 통해 가중치를 업데이트

5. 예측 및 평가

- 테스트 데이터를 사용해 예측 결과를 출력
- 실제 값과 비교하여 모델 성능을 평가

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# 데이터 준비 (생략)
data_lstm = pd.read_csv('your_data.csv')

# 모델 정의
model = Sequential([
    LSTM(units=50, return_sequences=True,
        input_shape=(X_train.shape[1], 1)),
    LSTM(units=50),
    Dense(units=1)
])

# 모델 컴파일
model.compile(loss='mean_squared_error', optimizer='adam')

# 모델 학습
model.fit(X_train, y_train, epochs=50, batch_size=16, verbose=1)

# 예측
y_pred = model.predict(X_test)

# 평가 (예: MAE)
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_test, y_pred)
print('MAE:', mae)
```

5. RNN과 LSTM

5.5 시계열 예측을 위한 RNN 및 LSTM 구조 ▶ 5.5.1 RNN 및 LSTM 예제 (1/5)

○ 데이터 준비

- ☑ 데이터 생성 : `generate_time_series_data` 함수는 샘플 개수와 시퀀스 길이에 따라 데이터 생성
- ☑ X 는 시퀀스의 마지막 데이터를 제외한 부분, y 는 마지막 데이터
- ☑ RNN과 LSTM은 3차원 입력을 필요로 하므로 (samples, timesteps, feature) 형태로 데이터를 변환
- ☑ 데이터 훈련용(80%)과 테스트용(20%)으로 분리

```
# 1. 시계열 데이터 생성
# 데이터:  $y = \sin(x)$ 
def generate_time_series_data(timesteps=100, samples=1000):
    x = np.linspace(0, 50, timesteps) # 0부터 50까지 timesteps(100)개로 나눈 값
    series = np.sin(x) + np.random.normal(scale=0.1, size=(samples, timesteps)) #  $\sin(x)$  + 노이즈
    return series

timesteps = 50
series = generate_time_series_data(timesteps=timesteps)

# 입력 데이터 (X): timesteps-1개의 데이터, 타겟 데이터 (y): 마지막 데이터
X = series[:, :-1] # 마지막 값을 제외한 데이터
y = series[:, -1] # 마지막 값이 타겟

# 데이터 차원 변환 (RNN/LSTM 입력 형식: (samples, timesteps, features))
X = X.reshape((X.shape[0], X.shape[1], 1)) # features=1, 입력 데이터 형태 맞추기

# 훈련 및 테스트 데이터 분리
train_size = int(len(X) * 0.8) # 80%는 훈련 데이터, 20%는 테스트 데이터
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
```


5. RNN과 LSTM

5.5 시계열 예측을 위한 RNN 및 LSTM 구조 ▶ 5.5.1 RNN 및 LSTM 예제 (2/5)

○ 모델 정의

☑ RNN 모델

- SimpleRNN(10)은 10개의 유닛으로 구성된 단순 RNN 계층
- activation='tanh'은 활성화 함수로, 데이터를 스케일링하여 안정적인 학습을 가능하게 함
- input_shape=(timesteps - 1, 1)은 시퀀스 길이와 피처수를 지정함

☑ LSTM 모델

- RNN 대신 LSTM(10)을 사용해 장기 의존성을 더 잘 처리할 수 있음
- Dense(1)은 단일 값을 출력하는 출력층. 이는 시계열 데이터의 다음 값을 예측

```
# 2. 모델 정의 (RNN)
rnn_model = Sequential([
    SimpleRNN(10, activation='tanh', input_shape=(timesteps - 1, 1)),
    Dense(1) # 출력값 (단일 값 예측)
])

# 모델 정의 (LSTM)
lstm_model = Sequential([
    LSTM(10, activation='tanh', input_shape=(timesteps - 1, 1)),
    Dense(1) # 출력값 (단일 값 예측)
])
```

5. RNN과 LSTM

5.5 시계열 예측을 위한 RNN 및 LSTM 구조 ▶ 5.5.1 RNN 및 LSTM 예제 (3/5)

○ 모델 정의

- ☑ 모델 컴파일 함수(model.compile)는 모델 학습 시 필요한 설정을 정의
- ☑ optimizer='adam'은 적응형 최적화 알고리즘으로, 학습 속도와 안정성을 높임
- ☑ loss='mse'는 평균 제곱 오차(MSE)를 손실 함수로 사용. 이는 예측값과 실제값의 차이를 최소화

```
# 3. 모델 컴파일
for model in [rnn_model, lstm_model]:
    model.compile(optimizer='adam', loss='mse')
```

○ 모델 컴파일

- ☑ 모델 컴파일 함수(model.compile)는 모델 학습 시 필요한 설정을 정의
- ☑ optimizer='adam'은 적응형 최적화 알고리즘으로, 학습 속도와 안정성을 높임
- ☑ loss='mse'는 평균 제곱 오차(MSE)를 손실 함수로 사용. 이는 예측값과 실제값의 차이를 최소화

```
# 4. 모델 학습
print("Training RNN model...")
rnn_model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

print("Training LSTM model...")
lstm_model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)
```

5. RNN과 LSTM

5.5 시계열 예측을 위한 RNN 및 LSTM 구조 ▶ 5.5.1 RNN 및 LSTM 예제 (4/5)

○ 모델 학습

- ☑ `model.fit` 함수로 모델을 학습함
- ☑ `epoch=10`은 데이터를 10번 반복하여 학습. `epoch`를 조정하면서 성능 개선
- ☑ `batch_size=32`는 한 번의 학습에 사용할 데이터 샘플 수를 의미. 작으면 학습이 세밀/느려짐, 크면 속도가 덜 세밀/빠름
- ☑ `verbose = 0`은 학습 중 출력 메시지 생략, `verbose = 1`은 학습 중 출력 메시지를 볼 수 있음

```
# 4. 모델 학습
print("Training RNN model...")
rnn_model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

print("Training LSTM model...")
lstm_model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)
```

○ 예측 및 평가

- ☑ `model.predict` 함수로 테스트 데이터를 사용해 예측 값을 생성
- ☑ 각 모델의 출력은 테스트 데이터의 타겟 값과 비교하기 위해 저장됨

```
# 5. 예측 및 평가
rnn_predictions = rnn_model.predict(X_test)
lstm_predictions = lstm_model.predict(X_test)
```

5. RNN과 LSTM

5.5 시계열 예측을 위한 RNN 및 LSTM 구조 ▶ 5.5.1 RNN 및 LSTM 예제 (5/5)

○ 결과 시각화

- ☑ plt.plot을 사용해 실제 값과 예측 값을 비교하여 시각화
- ☑ 첫 번째 서브플롯은 RNN 결과, 두 번째는 LSTM 결과를 나타냄
- ☑ 예측 값이 실제 값과 얼마나 잘 일치하는지 확인할 수 있음

```
# 6. 결과 시각화
plt.figure(figsize=(14, 5))

# 실제 값과 RNN 예측 비교
plt.subplot(1, 2, 1)
plt.plot(y_test[:100], label="Actual", color="blue")
plt.plot(rnn_predictions[:100], label="RNN Prediction", color="orange")
plt.title("RNN Predictions")
plt.legend()
plt.show

# 실제 값과 LSTM 예측 비교
plt.subplot(1, 2, 2)
plt.plot(y_test[:100], label="Actual", color="blue")
plt.plot(lstm_predictions[:100], label="LSTM Prediction", color="green")
plt.title("LSTM Predictions")
plt.legend()
plt.show()
```

6. 생성적 적대 신경망(GAN)

6.1 GAN의 기본 개념

- ☑ GAN(Generative Adversarial Networks)은, **생성자(Generator)**와 **판별자(Discriminator)**가 서로 경쟁하며 데이터를 **생성하는 모델**
- ☑ 생성자는 실제와 유사한 데이터를 생성하는 역할을 하며, 판별자는 주어진 데이터가 실제인지 가짜인지 구별하는 역할을 함

GAN 특징

- 높은 생성 능력 : 실제와 거의 구별할 수 없는 이미지, 텍스트, 음성, 동영상을 생성할 수 있음
- 효율적인 학습 : 생성기와 판별기의 경쟁을 통해 효율적으로 학습
- 적은 데이터로도 학습 가능 : 생성 모델을 활용하여 부족한 데이터를 보충할 수 있음

GAN 작동원리

- 판별자(판별기)는 입력 데이터가 실제데이터인지 모조데이터인지 구별하는 역할 수행
- 생성자(생성기)는 산출한 모조데이터를 판별기가 50% 확률로 진위 여부를 판단한다면 학습 종료



6. 생성적 적대 신경망(GAN)

6.2 GAN의 사용 예시 및 장단점

사용 예시

- 이미지 생성 : 예술작품, 사람 얼굴(딥페이크), 만화 캐릭터 생성
- 데이터 증강 : 의료 영상 데이터 부족 문제를 해결하기 위해 새로운 이미지를 생성
- 음성합성 : 실제 사람처럼 들리는 음성 합성
- 비디오 생성 : 특정 스타일의 비디오 클립 생성
- 스타일 변환 : 사진을 특정 그림 스타일(예: 피카소 스타일)로 변환
- 게임 개발 : 게임 환경이나 캐릭터 자동 생성

장점

- 실제와 유사한 데이터 생성 : 매우 사실적인 이미지나 데이터를 생성할 수 있음
- 창의적 활용 가능 : 새로운 예술 작품, 음악, 영상 제작에 활용 가능
- 데이터 부족 문제 해결 : 새로운 데이터를 생성하여 학습 데이터를 보강할 수 있음

단점

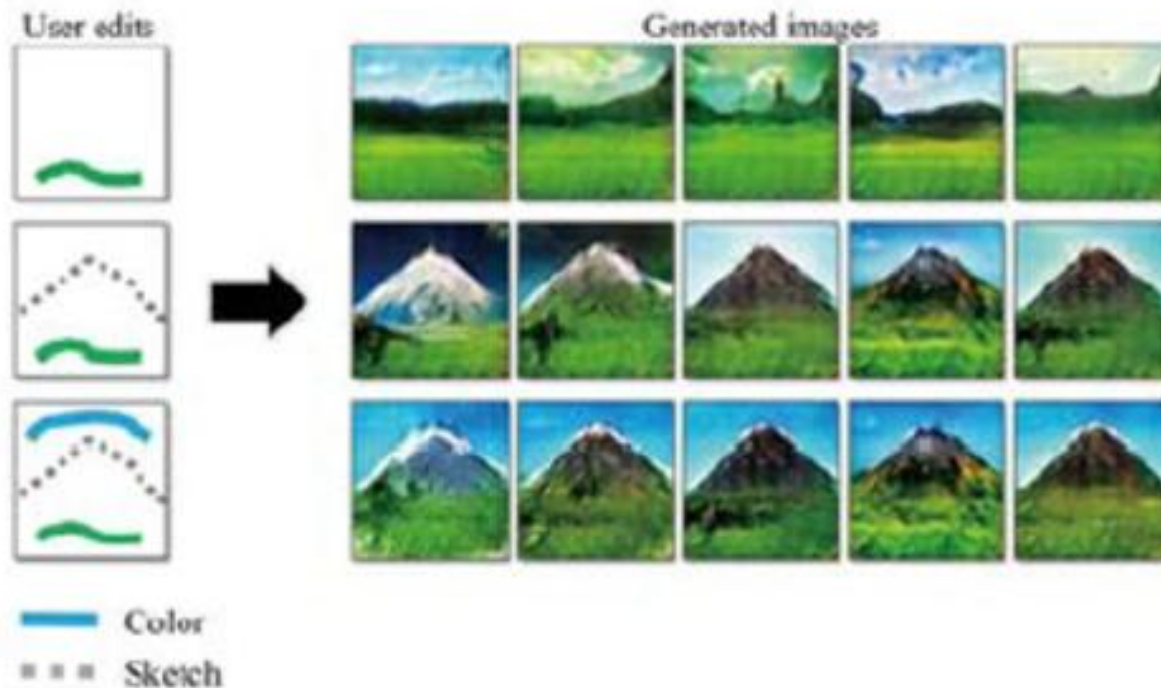
- 훈련 어려움 : 생성자와 판별자의 학습 균형을 맞추는 것이 어려움
- 윤리적 문제 : 딥페이크 등 악용 가능성이 존재
- 계산 비용 : 훈련 과정이 복잡하고 시간이 오래 걸릴 수 있음

6. 생성적 적대 신경망(GAN)

6.3 GAN의 응용 분야와 적용 사례 (1/4)

○ GAN을 활용한 새로운 이미지 생성

- ☑ 이미지 생성 인터페이스인 iGAN(Interactive Image Generation via GAN)을 통한 간단한 스케치로 이미지를 자동 생성

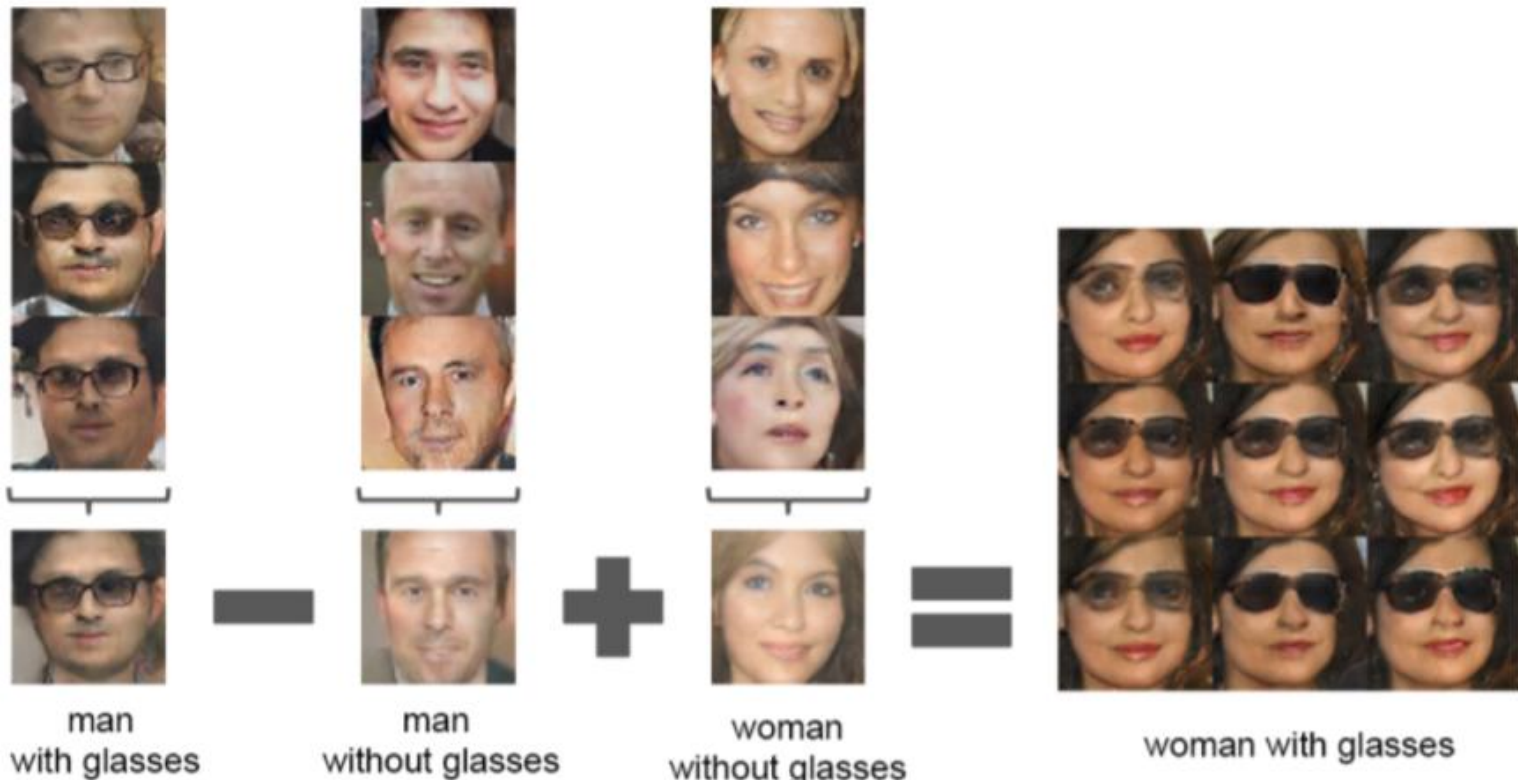


6. 생성적 적대 신경망(GAN)

6.3 GAN의 응용 분야와 적용 사례 (2/4)

○ DCGAN(Deep Convolutional GAN)

- ☑ 모델에서 생성된 이미지 중 안경 낀 남자, 안경 안 낀 남자, 안경 안 낀 여자 그룹을 만들고 각 그룹에서 잠재공간(latent vector)의 평균을 구한 값으로 안경 안 낀 여자 이미지 결과값 도출



6. 생성적 적대 신경망(GAN)

6.3 GAN의 응용 분야와 적용 사례 (3/4)

○ SRGAN(Supers Resolution GAN)

- ☑ 화질이 선명하지 않은 사진을 GAN을 활용하여 복원하는 기능
- ☑ 미세한 질감 데이터의 복구를 기반으로 만들어짐 (저해상도 이미지 → 고해상도 이미지)



6. 생성적 적대 신경망(GAN)

6.3 GAN의 응용 분야와 적용 사례 (4/4)

○ StackGAN

- ☑ 입력된 문장과 단어를 해석해 이미지를 생성



6. 생성적 적대 신경망(GAN)

6.4 GAN 구조 및 고려사항

GAN 기본 구조

1. 데이터 준비 (Input)

- GAN이 학습할 데이터를 준비

2. 모델 정의 (Model)

- 생성자와 판별자 모델을 정의

3. 모델 컴파일

- 판별자와 GAN(생성자 + 판별자 결합) 모델을 각각 컴파일

4. 모델 학습

- 진짜와 가짜 데이터를 학습하여 생성자가 더 좋은 데이터를 만들도록 훈련

5. 예측 및 평가

- 생성자가 만든 데이터를 확인하고 성능을 평가

고려사항

데이터 전처리

- 이미지 데이터의 경우 크기조절, 정규화 등 전처리 과정을 거쳐야 함

하이퍼 파라미터 설정

- 학습률, 배치 크기, epoch 수 등 적절하게 설정

학습 불안정 시 문제 해결

- 하이퍼파라미터 조정, 모델 아키텍처 변경, 데이터 증강 등 다양한 방법 시도

활성화 함수

- LeakyReLU, Tanh 등 다양한 활성화 함수를 사용하여 모델의 표현력 증대

생성자와 판별자 학습 균형

- 생성자와 판별자를 번갈아가며 학습

7. 머신러닝 및 딥러닝 분류 비교

- ☑ 머신러닝의 분류 알고리즘(SVM, 로지스틱회귀 등)은 특징(피쳐)을 사람이 직접 지정, CNN은 필요한 특징을 자동으로 학습
- ☑ CNN은 딥러닝 모델이므로 딥러닝의 범주에 속하여, TensorFlow나 PyTorch 같은 딥러닝 라이브러리를 사용함

구분	머신러닝의 분류	딥러닝의 분류 (CNN 등)
주요특징	- 입력데이터에서 직접 특징(feature)을 추출해야 함 - 비교적 간단한 알고리즘으로 작동	- 자동으로 특징(feature)을 학습 - 복잡한 데이터를 처리 가능(예: 이미지, 비디오)
데이터 처리 방식	주어진 특징에 기반한 수학적 모델 사용 (예 : SVM, 로지스틱 회귀)	인공신경망을 사용하여 데이터를 학습
필요한 데이터 양	상대적으로 적은 데이터로도 학습 가능	대규모 데이터가 필요. 특히 CNN은 이미지 데이터에서 강력
연산 리소스	CPU만으로도 충분한 경우가 많음	GPU/TPU와 같은 고성능 연산 장치가 필요
적용 예시	스팸 메일 필터링, 고객 분류 등 간단한 분류 작업	이미지 분류, 음성 인식, 자율 주행차 데이터 분석 등